

תודות

אנו מעוניינים להודות תחילה למנחה הפרויקט שלנו משה גואטה שהשקיע מזמנו ועזר לנו בכל התהליך מתחילתו ועד סופו. משה העניק לנו כלים ללמידה ולכישורי פעולה בצוות.

נוסף אנו מעוניינים להודות לראש המחלקה לאלקטרוניקה, אייל קודלר על התמיכה לאורך הדרך.

ברצוננו להודות לצחי קובל האגרונום שייעץ בנושא השקייה, טנסיומטרים ומחזורי תאורה לצמחים.

ברצוננו להודות ליחזקאל המסגר שעזר לנו לבנות את גג החממה ואת התריס.

לסיכום, ברצוננו להודות לסבא של אילון על העזרה בבניית דגם החממה מתחילתו ועד סופו.

הצעת הפרויקט

חממה חכמה הכוללת : בקרת טמפ', השקיה אוטומטית של הצמח בהתאם ללחות האדמה ע"י טנסיומטר (סנסור לבדיקת לחות האדמה), בקרת אוויר, שליטה על מחזור האור והחושך בחממה ע"י תריסים/וילונות ותאורה בתוך החממה.

רשימת רכיבים

בקרה ומתח

- מעגל Arduino Mega
- ספק כח למנועים 12v/4A
- מפסק on/off

מערכת אוורור

- 4 מאווררים
- 4 ממסרים
- 2 חיישני טמפרטורה

מערכת השקיה אוטומטית

הצד של הארדואינו

- XBee Arduino Shield
- XBee
- ברז חשמלי DC Latch
- מיכל מים וצנרת מים לצמח – השקיה בגרביטציה

הצד של הצמח

- סוללה 9v
- XBee
- טנסיומטר – מד לחות אדמה
- פוטנסיומטר
- מפסק on/off ולד אינדיקציה
- לד אינדיקצית שידור
- מפסק בורר בין הטנסיומטר לפוטנסיומטר

מערכת כניסה ויציאה

- מטריצת לחצנים 4x4
- דרייבר
- דלת כוון דיסקים של מחשב המשמש כדלת החממה
- כפתור לפתיחת הדלת מתוך החממה

מערכת בקרת תאורה

- LDR – פוטורזיסטור
- סרט לדים
- ממסר
- מנוע תריס
- דרייבר

פירוט דרישות מהמבצע

1. לימוד החומר.
2. תכנון המערכת.
3. הרכבת המערכת.
4. אינטגרציה של המערכת.
5. מדידות ואיתור תקלות.
6. כתיבת ספר הפרויקט.

תוכן עניינים

11	1 מבוא
11	1.1 תיאור המערכת
11	1.2 עיקרון הפעולה
12	2 קיצורים
13	3 תכנון המערכת
13	3.1 תרשים מלבנים
13	3.1.1 דיאגרמה כללית
14	3.1.2 מערכת השקייה
15	3.1.3 מערכת תאורה
16	3.1.4 בקרת כניסה
17	3.1.5 בקרת טמפרטורה
18	3.2 סכמה חשמלית
18	3.2.1 מעגל השקייה – צד ארדואינו
19	3.2.2 מעגל השקייה – מעגל הצמח
20	3.2.3 מעגל בקרת כניסה
21	3.2.4 מעגל מאווררים
22	3.2.5 מעגל בקרת תאורה
23	3.2.6 מעגל חיישני חממה
24	3.3 פינים ארדואינו
25	3.4 פינים מחבר DB25
26	3.5 תיאור רכיבים
26	3.5.1 ארדואינו Mega
26	3.5.2 ספק כוח
26	3.5.3 מאוורר
27	3.5.4 ממסר
28	3.5.5 חיישן טמפרטורה
29	3.5.6 XBee S1 802.15.4 RF Module
34	3.5.7 ברז חשמלי
35	3.5.8 סוללת 9V
35	3.5.9 מייצב מתח מעגל Pololu S7V8F3
35	3.5.10 טנסיומטר
36	3.5.11 לוח מקשים 4x4
37	3.5.12 דרייבר מנוע HW-95
37	3.5.13 כונן דיסקים
38	3.5.14 LDR

38	3.5.15 סרט לדים
38	3.5.16 מנוע תריס
38	3.5.17 ממשק טורי UART
39	4 מדידות
39	4.1 מדידות טנסיומטר
39	4.2 חישוב מתח סוללה ו-% שנותר
39	4.3 חישוב טיימר 1sec
40	4.3.1 מדידת חיישן טמפרטורה
40	4.3.2 מדידות RSSI
40	4.3.3 מדידת הזרם הנצרך מספק הכוח של 12V
41	5 תיאור תכנה
41	5.1 תכנת GreenDo
41	5.1.1 פונקציות לטיפול בשעון זמן
41	5.1.2 פונקציות כיבוי והפעלת מערכות בחממה
43	5.1.3 פונקציות טיימרים תוכנתיים
44	5.1.4 פונקציות ממשק בין תוכנת GreenSee ל- GreenDo
46	5.1.5 פונקציות לתקשורת עם XBee
48	5.1.6 פונקציות דלת כניסה
49	5.1.7 פונקציות לבקרת טמפרטורה
50	5.1.8 פונקציות לבקרת תאורה
53	5.2 פלט קוד GreenDo
67	5.3 אפליקציית GreenSee
67	5.3.1 MainWindow
67	5.3.2 ParseMsg
67	5.3.3 GreenLineStateMachine
68	5.3.4 MainTimerTick
68	5.3.5 ConfigurationButton_Click
69	5.4 פלט תוכנת GreenSee
74	6 איתור תקלות וקשיים בפרויקט
74	6.1 איתור תקלות
74	6.1.1 Timer Interrupt
74	6.1.2 תוכנה נתקעה
74	6.1.3 פונקציית delay() אינה עובדת בתוך פסיקה
74	6.1.4 Big/Little-endian
75	6.1.5 חיישן טמפרטורה מציג ערכים לא נכונים
75	6.1.6 קפיצה בנתון הטמפרטורה

77	6.2 קשיים בפרויקט
77	6.2.1 הלחמה ו-wire-wrap
77	6.2.2 לימוד רכיב XBee
77	6.2.3 בניית דגם החממה
77	6.2.4 פסיקות, טיימר תוכנתי ותוכנת GreenSee
77	6.2.5 אינטגרציה של מערכות
78	7 ביבליוגרפיה
79	8 נספח א : תיעוד פגישות פרויקט

רשימת תרשימים

13	תרשים 1 : דיאגרמת מלבנים כללית
14	תרשים 2 : דיאגרמת מלבנים של מערכת השקייה
15	תרשים 3 : דיאגרמת מלבנים של מערכת בקרת התאורה
16	תרשים 4 : דיאגרמת מלבנים של מערכת בקרת הכניסה
17	תרשים 5 : דיאגרמת מלבנים של המערכת לבקרת הטמפרטורה
18	תרשים 6 : סכמה חשמלית - מעגל השקייה (הצד של הארדואינו)
19	תרשים 7 : סכמה חשמלית של מעגל הצמח
20	תרשים 8 : סכמה חשמלית של מערכת בקרת הכניסה
21	תרשים 9 : סכמה חשמלית של המאווררים
22	תרשים 10 : סכמה חשמלית של מעגל התאורה
23	תרשים 11 : סכמה חשמלית של חיישני החממה
25	תרשים 12 : מחבר DB25
27	תרשים 13 : מאוורר יונק חום מהחממה
27	תרשים 14 : תמונת המאוורר שבו השתמשנו
28	תרשים 15 : סכמה חשמלית של ממסר
28	תרשים 16 : תיאור פינים של חיישן הטמפרטורה
29	תרשים 17 : גרף מתח כפונקציה של הטמפרטורה
30	תרשים 18 : פורמט הודעת API כללית
31	תרשים 19 : פורמט API של פקודת AT Command Frame
32	תרשים 20 : מסך שינוי רגיסטרים בתוכנת XCTU
33	תרשים 21 : הודעת RX Packet IO (16bit)
34	תרשים 22 : פורמט RF Data בהודעת IO
34	תרשים 23 : ברז חשמלי
35	תרשים 24 : טנסיומטר אנלוגי
37	תרשים 25 : דרייבר מנוע
38	תרשים 26 : פורמט UART
41	תרשים 27 : תרשים זרימה כללי GreenDo
46	תרשים 28 : מכונת מצבים קליטת הודעה מ-GreenSee
47	תרשים 29 : מכונת מצבים לקליטת הודעה
49	תרשים 30 : תרשים זרימה בקרת כניסה
51	תרשים 31 : מעגל ה-LDR
52	תרשים 32 : תרשים זרימה בקרת תאורה
67	תרשים 33 : צילום מסך של תוכנת GreenSee

רשימת טבלאות

24	טבלה 1 : מספרי הפינים בארדואינו
25	טבלה 2 : פינים של מחבר DB25
31	טבלה 3 : ערכי רגיסטרים ב- XBee בצד של הארדואינו
32	טבלה 4 : ערכי רגיסטרים ב- XBee בצד של הצמח
33	טבלה 5 : פינים בשימוש ב- XBee צמח
37	טבלה 6 : פינים בדרייבר בשימוש בפרויקט
39	טבלה 7 : מדידות מתח טנסיומטר
40	טבלה 8 : זרם נצרך במערכות
44	טבלה 9 : פרוטוקול תקשורת בין תכנת GreenSee ל- GreenDo
44	טבלה 10 : הודעות מ- GreenDo ל- GreenSee
45	טבלה 11 : הודעות מ- GreenSee ל- GreenDo
51	טבלה 12 : טבלת המרה מערך A/D ל- LUX
79	טבלה 13 : תיעוד פגישות פרויקט

1 מבוא

1.1 תיאור המערכת

החממה הינה חממה בעלת מערכות בקרה אוטומטיות אשר מספקות לצמחים בחממה תנאים אופטימליים ע"י ויסות טמפרטורה, שינוי מחזוריות אור/חושך, בקרת השקייה ע"י חישה אלחוטית של מד יובש האדמה ומערכת בקרת כניסה לחממה ע"י קודן.

הפרויקט כולל אפליקציית ממשק משתמש לשליחת הגדרות ותכניות עבודה לבקר ותצוגה של מצב החיישנים בחממה.

1.2 עיקרון הפעולה

הבקרה מבוססת על מעגל Arduino Mega השולט על המערכות השונות ע"י כניסות/יציאות דיגיטליות ואנלוגיות בצורה ישירה או ע"י דרייברים וממסרים. הבקר מחובר לרכיב XBee Coordinator אשר מחובר בתקשורת RF ל-XBee End device שנמצא במקום מרוחק מהבקר ומחובר לחיישן מד יובש האדמה. ממשק המשתמש מבוסס אפליקציית WPF שמחוברת לבקר בערוץ טורי.

קיצורים 2

ADC – Analog to Digital Converter

AES – Advanced Encryption Standard

API - Application Programming Interface

ASCII - American Standard Code For Information Interchange

LDR - Light Dependent Resistor

PAN – Personal Area Network

PWM – Pulse Width Modulation

RF – Radio Frequency

WPF – Windows Presentation Foundation.

מודל וכלי פיתוח תוכנה של מיקרוסופט למערכת ההפעלה Windows עם דגש על עיצוב גרפי מתקדם של ממשק המשתמש.

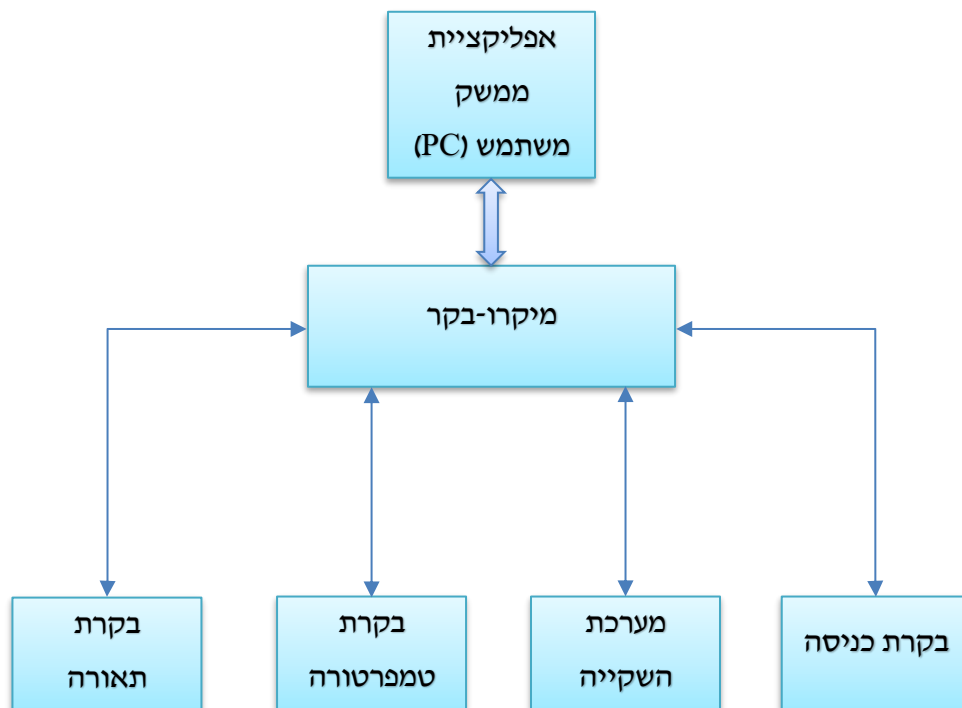
3 תכנון המערכת

המערכת בוססה על תפקודה של חממה רגילה (השקיית הצמחים, בקרת טמפרטורה ובקרת תאורה). בניגוד לחממות הקיימות כעת בשוק, תכננו את החממה כך שבקורות אלו יהיו אוטומטיות, כך שהתערבות המשתמש תהיה כמה שיותר מינימלית.

3.1 תרשימי מלבנים

3.1.1 דיאגרמה כללית

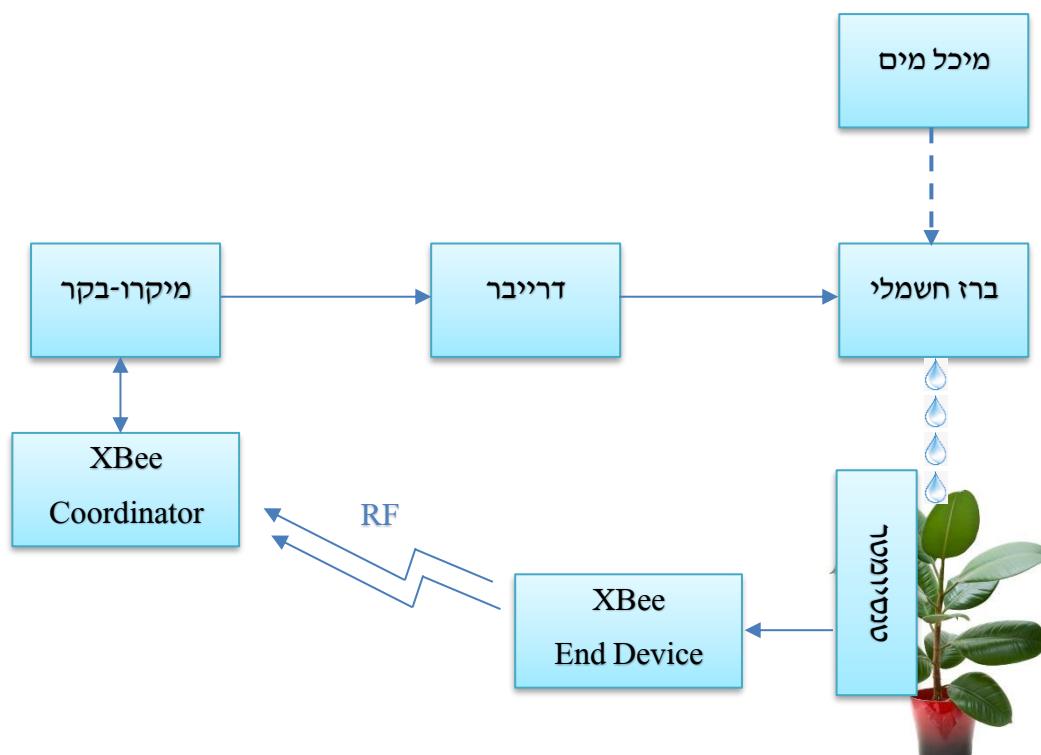
המערכת מבוססת על מיקרו-בקר המיושם ב Arduino Mega, המיקרו-בקר מקבל קונפיגורציה מאפליקציית GreenSee, אשר גם מציגה סטאטוס של חיישנים בחממה. ממשק בין המחשב לבקר הוא ממשק טורי באמצעות USB. המיקרו-בקר שולט בתתי המערכות השונות אשר כל אחת מהן בלתי תלויה בשנייה. סכמת מלבנים של כל תת מערכת מופיע בהמשך. כל המערכות שצריכות הספק גבוה כגון מנועים, תאורה וברז חשמלי מקבלים מתח עבודה של 12v מספק כח חיצוני.



תרשים 1: דיאגרמת מלבנים כללית

3.1.2 מערכת השקייה

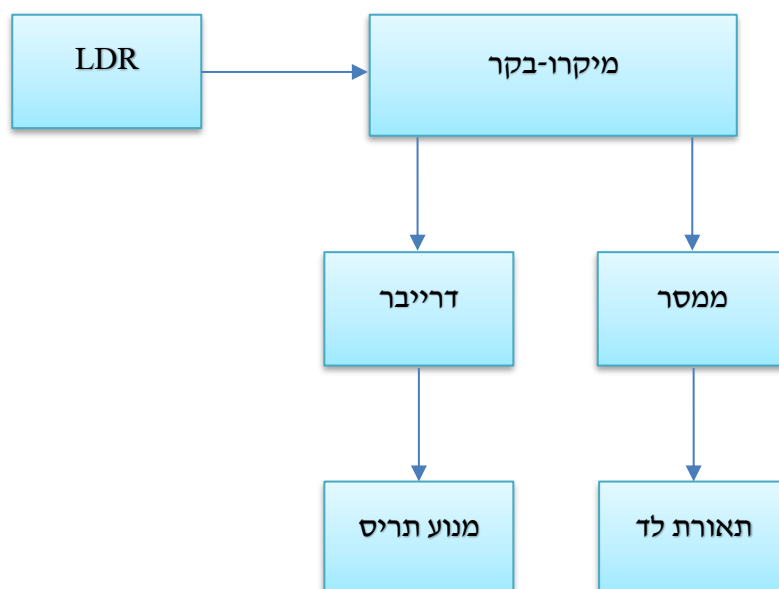
רכיב ה-XBee בצד של הצמח מוגדר כ- End Device ומשדר ב- RF כל 5 שניות נתון אנלוגי של מתח הטנסיומטר ומתח מצב הסוללה במעגל הצמח. רכיב ה-XBee Coordinator קולט את השידור ומעביר את ההודעה למיקרו-בקר בתקשורת טורית. המיקרו-בקר מפענח את ההודעה ובהתאם למתח הטנסיומטר ותוכנית ההשקייה השבועית, פותח או סוגר ברז מים חשמלי. פתיחת הברז מזרימה מים לצמח למשך זמן המוגדר בתוכנית ההשקייה. המים זורמים בגרביטציה ממכל המדמה מאגר מים אל הצמח באמצעות צנרת מים.



תרשים 2: דיאגרמת מלבנים של מערכת השקייה

3.1.3 מערכת בקרת תאורה

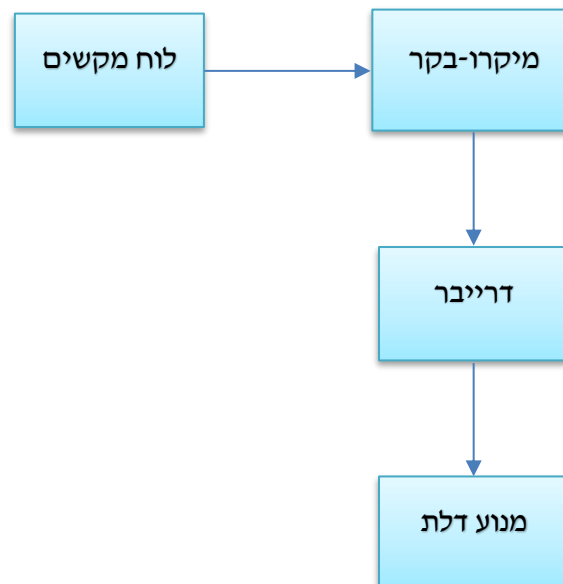
במיקרו-בקר מוגדרת תכנית תאורה שבועית, אשר מגדירה באיזה שעות צריך אור ובאילו שעות צריך להחשיך. המיקרו-בקר מודד את רמת האור בחממה, אם צריך להחשיך המיקרו-בקר מכבה את האור ומוריד את הווילון. אם צריך להאיר המיקרו-בקר מעלה את הווילון, במידה ורמת התאורה מבחוץ אינה מספיקה המיקרו-בקר מדליק את מנורות תאורת הלד בחממה.



תרשים 3 : דיאגרמת מלבנים של מערכת בקרת התאורה

3.1.4 בקרת כניסה

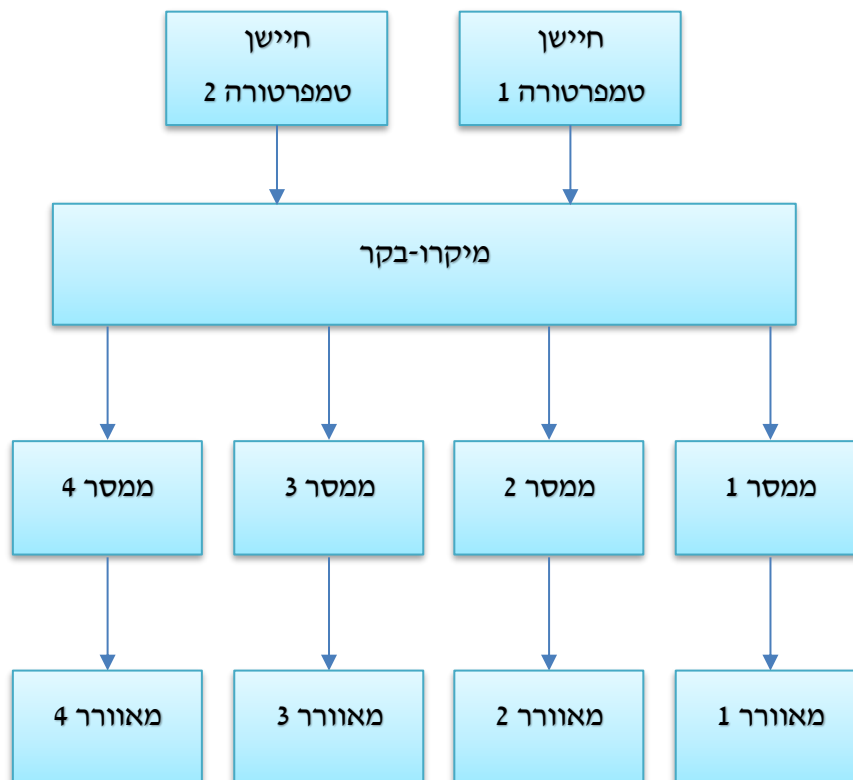
על מנת לפתוח את דלת החממה, המשתמש צריך להקיש קוד סודי של 4 ספרות. לאחר הקשת הקוד המיקרו-בקר פותח את הדלת וסוגר אותה לאחר השהייה של 10 שניות. במידה והוקש קוד שגוי, במשך שלום פעמים, המערכת ננעלת ולא ניתן לפתוח את הדלת באמצעות לוח המקשים למשך חצי דקה. דרך נוספת לפתיחת הדלת היא ע"י לחצן שנמצא בתוך החממה.



תרשים 4: דיאגרמת מלבנים של מערכת בקרת הכניסה

3.1.5 בקרת טמפרטורה

המיקרו-בקר דוגם את הטמפרטורה משני חיישני הטמפרטורה פעם בשנייה וממצע ביניהם. למיקרו-בקר מוגדר טווח טמפרטורות רצוי לצמח. במידה והטמפרטורה הממוצעת גבוהה מהערך העליון של טווח הטמפרטורות, המיקרו-בקר מפעיל 4 מאווררים הפזורים ברחבי החממה. במידה והטמפרטורה הממוצעת נמוכה מהערך התחתון של טווח הטמפרטורות, המיקרו-בקר מכבה את המאווררים.

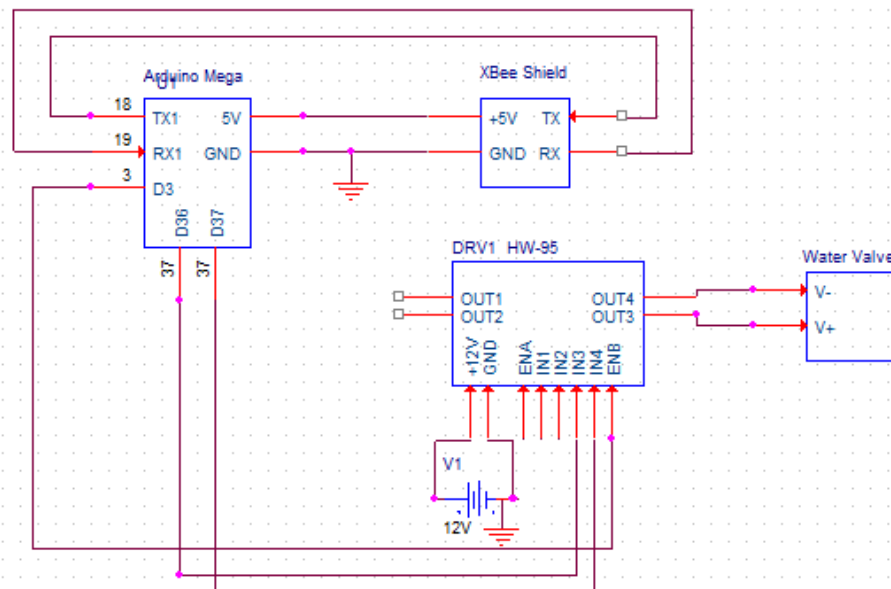


תרשים 5: דיאגרמת מלבנים של המערכת לבקרת הטמפרטורה

3.2 סכמות חשמליות

ספק כח 12V המספק מתח למערכות השונות בחממה. רגל '-' של הספק, מחוברת ל-GND של הארדואינו, על מנת שיהיה לכל המערכות אותו מתח ייחוס.

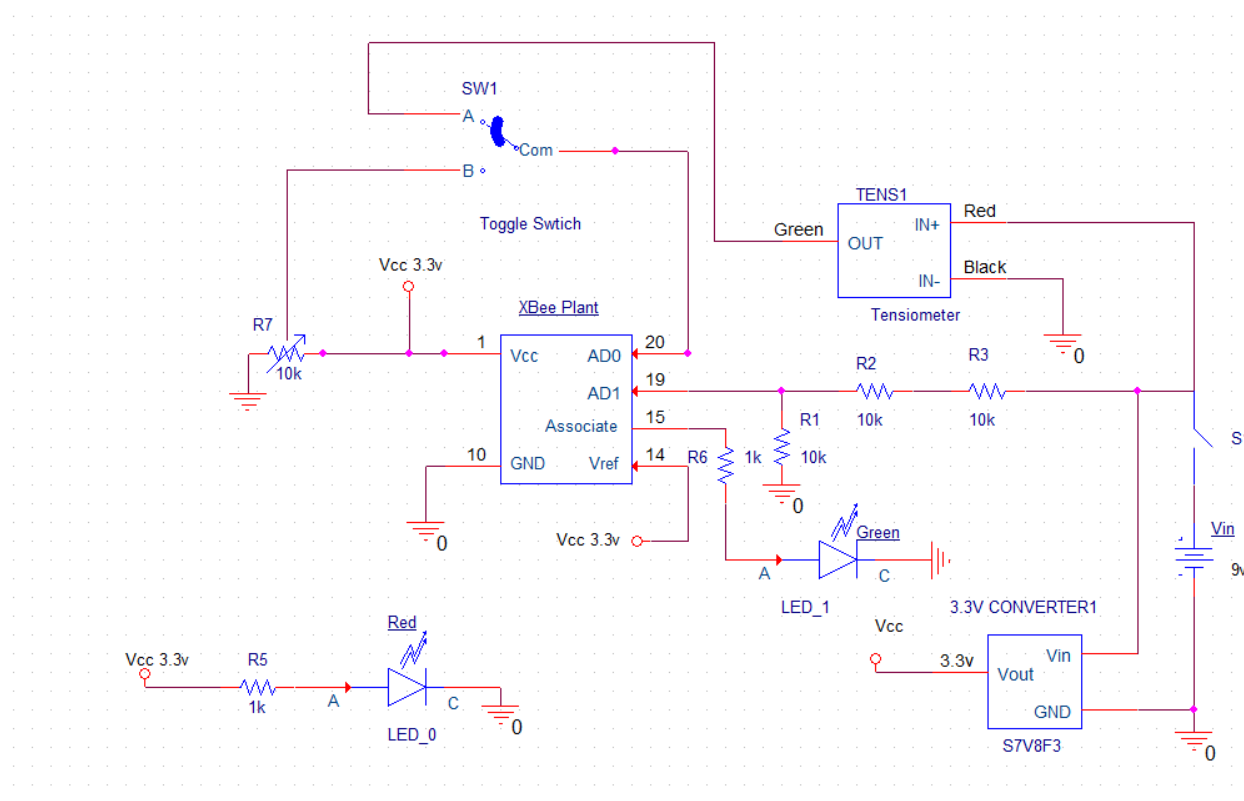
3.2.1 מעגל השקייה – הצד של הארדואינו



On the XBee Shield the switch has to be on UART mode.

תרשים 6: סכמה חשמלית - מעגל השקייה (הצד של הארדואינו)

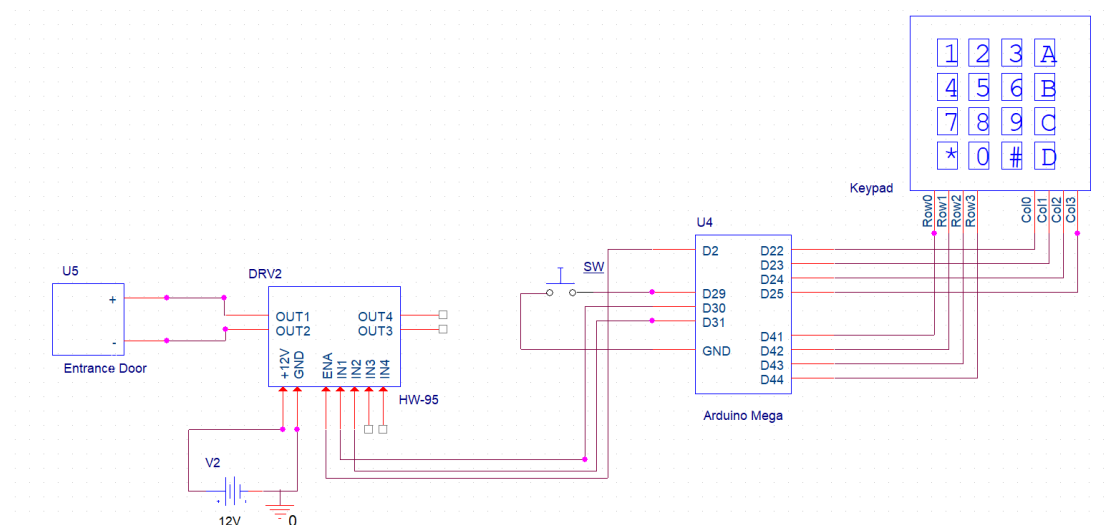
ה-XBee מחובר לארדואינו דרך ה-XBee Shield שאליו מחוברות רגלי ה-RX וה-TX של הארדואינו. ה-XBee Shield מקבל 5V מהארדואינו. ברז המים מחובר לארדואינו דרך דרייבר HW-95, הוא מחובר ליציאות OUT3 ו-OUT4 של הדרייבר. פורטים 3, 36 ו-37 של הארדואינו מחוברים בהתאמה לפינים ENB, IN3 ו-IN4. הדרייבר מקבל מתח מספק כוח חיצוני של 12V.



תרשים 7: סכמה חשמלית של מעגל הצמח

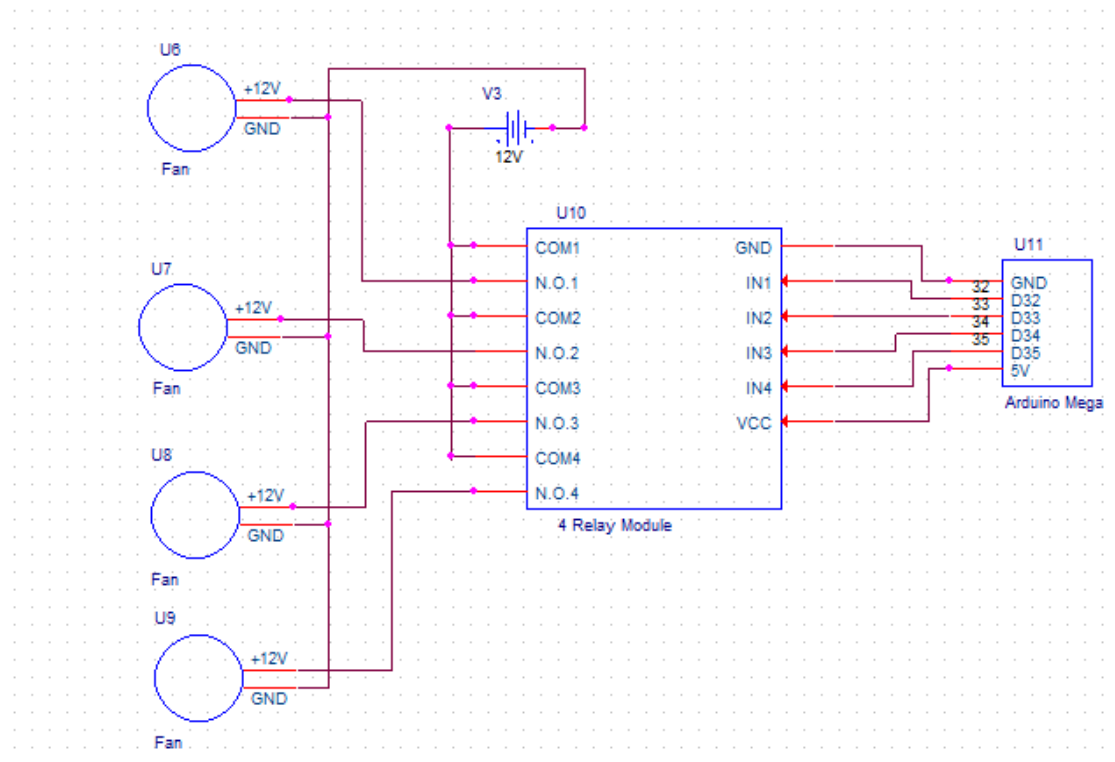
ה-XBee בצד של הצמח מקבל מתח של 3.3V מממיר המתח. בין רגל Associate של ה-XBee לאדמה מחובר LED ירוק שמהבהב כל פעם שה-XBee משדר הודעה. רגל Vref של ה-XBee מחוברת גם כן ל-3.3V מממיר המתח; זהו מתח הייחוס לדגימות A/D. יציאה OUT של הטנסיומטר מחוברת לרגל AD0 של ה-XBee, רגל IN- שלו מקוצרת לאדמה. בין רגל IN+ של הטנסיומטר לאדמה מחובר מפסק וסוללה של 9V. ממיר המתח מקבל 9V מהסוללה כאשר המפסק סגור ומספק 3.3V מיוצב ל-XBee. לפין AD1 נכנס המתח של הסוללה מחולק פי שלוש ע"י מחלק מתח. בין ה-Vcc לאדמה מחובר LED אדום לשם אינדיקציה שהמעגל עובד. בשלב מאוחר הוספנו מפסק שבורר בין מתח הטנסיומטר למתח הפוטנציומטר המדמה את פעולת הטנסיומטר וזאת לשם הדגמה של שינוי מתח הטנסיומטר שמשתנה מאוד לאט בפועל.

3.2.3 מעגל בקרת כניסה



תרשים 8: סכמה חשמלית של מערכת בקרת הכניסה

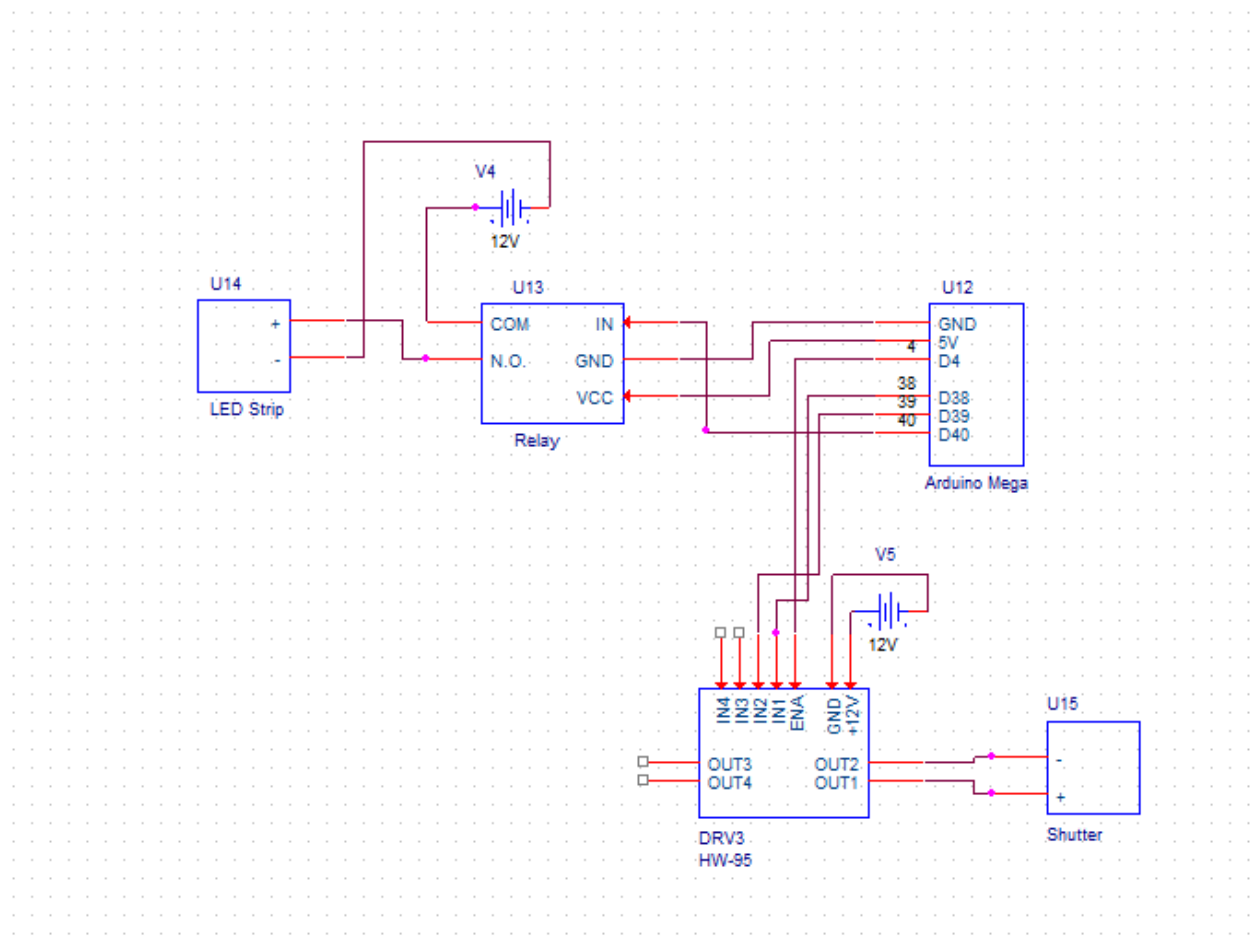
הארדואינו מחובר לפינים ENA, IN1 ו-IN2 של הדרייבר דרך הפורטים 2, 30 ו-31. לדרייבר מסופק מתח של 12V מספק כוח חיצוני. הדלת מחוברת ליציאות OUT1 ו-OUT2 של הדרייבר. בין פורט 29 של הארדואינו לאדמה מחובר לחצן לפתיחת הדלת מתוך החממה. לוח מקשים 4x4 מחובר לארדואינו, כאשר ארבעת הפורטים ROW0 עד ROW3 מחוברים בהתאמה לפורטים 41 עד 44 של הארדואינו וארבעת הפורטים COL0 עד COL3 מחוברים בהתאמה לפורטים 22 עד 25 של הארדואינו.



תרשים 9: סכמה חשמלית של המאווררים

פורטים 32 עד 35 של הארדואינו מחוברים בהתאמה לפינים IN1 עד IN4 של הממסר. ממסר מקבל מתח של 5V מהארדואינו. היציאות COM1 עד COM4 מחוברות כולן ל-12V של ספק הכוח החיצוני. ה-GND של ספק הכוח מחובר לכל רגל GND של כל אחד מארבעת המאווררים. היציאות N.O.1 עד N.O.4 (N.O. משמעותו Normally Open) מחוברות כל אחת בנפרד לרגל +12V של כל אחד מהמאווררים.

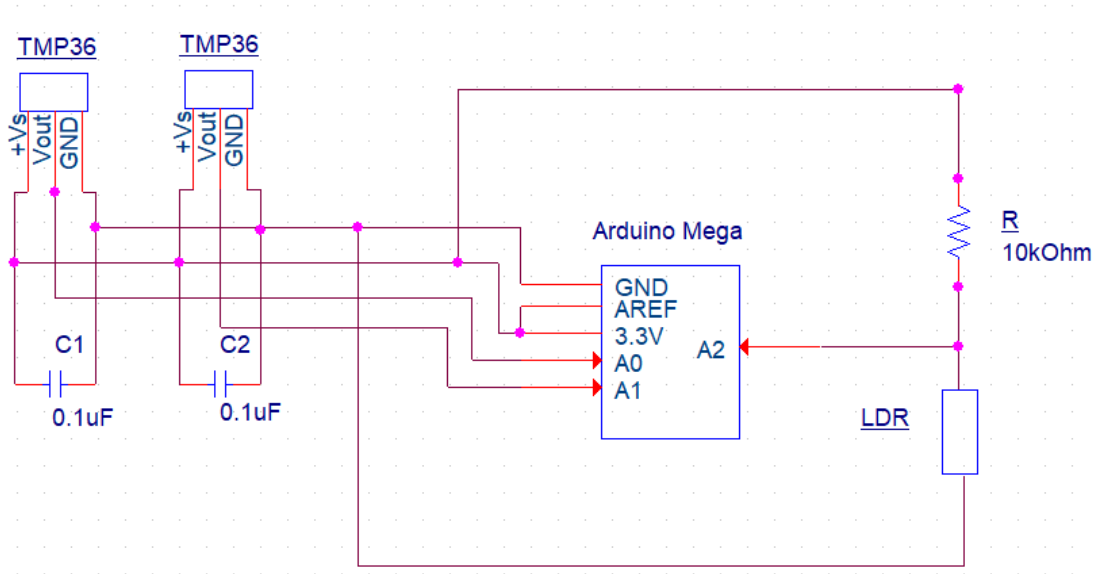
3.2.5 מעגל בקרת תאורה



תרשים 10: סכמה חשמלית של מעגל התאורה

ממסר מקבל מתח של 5V מהארדואינו. פורט 40 של הארדואינו מחובר לרגל IN של הממסר. יציאה N.O. של הממסר מחוברת לרגל '+' של סרט הלדים. לרגל COM מחובר ההדק החיובי של ספק הכוח. ההדק השלילי של ספק הכוח מחובר לרגל '-' של סרט הלדים.

הפורטים 4, 38 ו-39 של הארדואינו מחוברים בהתאמה לפינים ENA, IN1 ו-IN2 של הדרייבר. היציאות של הדרייבר OUT1 ו-OUT2 מחוברות לרגל '+' ולרגל '-' של מנוע התריס. הדרייבר מקבל מתח מספק כוח חיצוני.



תרשים 11: סכמה חשמלית של חיישני החממה

בין הרגליים +Vs ו-GND של כל אחד מחיישני הטמפרטורה מחובר קבל שערכו $0.1\mu\text{F}$ להפחתת רעשים. כל אחד מהחיישנים מקבל מתח של 3.3V מהארדואינו. היציאה Vout של אחד החיישן הראשון מחוברת לפורט האנלוגי A0 של הארדואינו והיציאה של החיישן השני מחוברת לפורט A1. ל-LDR מחובר בטור נגד שערכו $10\text{k}\Omega$. הנגד וה-LDR מחוברים בין 3.3V ל-GND של הארדואינו. המתח ממחלק המתח נכנס לפורט A2.

3.3 פיינים ארדואינו

הטבלה הבאה מפרטת את כל הפיינים שמחוברים לארדואינו בכל המעגלים

טבלה 1 : מספרי הפיינים בארדואינו

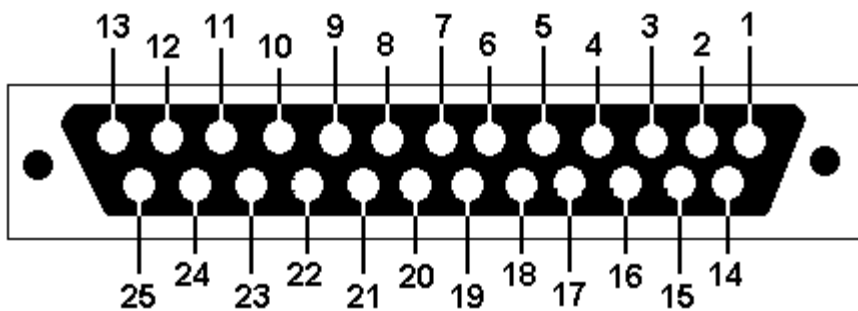
מס' רגל	סוג אות	שימוש
2	PWM out	ENA לדרייבר של דלת הכניסה
3	PWM out	ENB לדרייבר של ברז המים
4	PWM out	ENA לדרייבר של התריס
18	TX1	ל- TX ב- XBee Shield
19	RX1	ל- RX ב- XBee Shield
22	Digital in	COL0 ללוח המקשים
23	Digital in	COL1 ללוח המקשים
24	Digital in	COL2 ללוח המקשים
25	Digital in	COL3 ללוח המקשים
29	Digital in internal pullup	לחצן לפתיחת הדלת מתוך החממה
30	Digital out	IN1 לדרייבר של דלת הכניסה
31	Digital out	IN2 לדרייבר של דלת הכניסה
32	Digital out	IN1 לממסר של המאווררים
33	Digital out	IN2 לממסר של המאווררים
34	Digital out	IN3 לממסר של המאווררים
35	Digital out	IN4 לממסר של המאווררים
36	Digital out	IN3 לדרייבר של ברז המים
37	Digital out	IN4 לדרייבר של ברז המים
38	Digital out	IN1 לדרייבר של התריס
39	Digital out	IN2 לדרייבר של התריס
40	Digital out	IN לממסר של סרט הלדים
41	Digital in	ROW0 ללוח המקשים
42	Digital in	ROW1 ללוח המקשים
43	Digital in	ROW2 ללוח המקשים
44	Digital in	ROW3 ללוח המקשים
A0	Analog in	מחיישן טמפ' 1
A1	Analog in	מחיישן טמפ' 2
A2	Analog in	ממחלק מתח LDR

3.4 פנינים מחבר DB25

מחבר DB25 בין מעגל הבקרה של הארדואינו הנמצא מחוץ לחממה לבין המערכות בתוך החממה, לשם נוחות ניוד החלקים השונים של הפרויקט.

טבלה 2: פנינים של מחבר DB25

מס'	שימוש
3	מאוורר 3 +12V
5	מאוורר 2 +12V
7	מאוורר 1 +12V
9	לדים +12V
11	לדים GND
13	מאווררים GND
14	מאוורר 4 +12V
15	לחצן דלת – סיגנל חוזר
16	דלת +
17	לחצן דלת GND
18	דלת -
20	תריס +
22	תריס -



תרשים 12: מחבר DB25

3.5 תיאור רכיבים

3.5.1 ארדואינו Mega

ארדואינו הוא מחשב קטן שיכול להתממשק אל העולם הפיזי, הוא חלש יותר ממחשב ביתי אבל יש לו גישה אל רכיבים חיצוניים בניגוד למחשב הביתי. גם המחשב הביתי מחובר אל אמצעי קלט ופלט כמו מקלדת ומסך. בליבו של הארדואינו יושב בקר ATmega328, בקר זה אחראי על ביצוע כל הפקודות שאנחנו כותבים בסביבת הפיתוח ובשמירתן בזיכרון שלו. שפת התכנות של הארדואינו נותנת לנו גישה להפעיל ולכבות פורטים מסוימים במיקרו-בקר. בארדואינו מגה ישנם 56 פורטים דיגיטליים שהם הדרך שלנו להפעיל ולכבות את המתח שאותו אנחנו רוצים לספק לפלט כלשהו, או לקבל קלט ממקור כלשהו. הפורטים הם למעשה בסיס הלוגיקה של הארדואינו והמתח שבו הם עובדים הוא 5 וולט. ל-12 פינים מתוך ה-56 ישנה יכולת לספק מתח מ-0 עד 5 וולט ע"י PWM. בנוסף ישנם 16 פינים לכניסה אנלוגית המפענחים את המתח המשתנה שהם מקבלים וממירים אותו לערך דיגיטלי שבין 0 ל-1023.

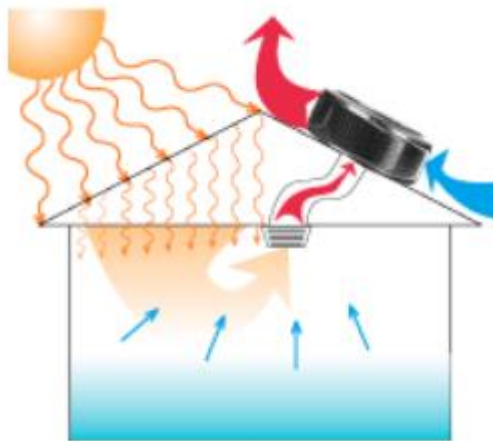
דבר אחד שחשוב לזכור הוא שהיכולת של הפינים הדיגיטליים היא מוגבלת בערך ל-40mA לכל פין. כשמדליקים לד אין עם זה כל בעיה, אבל אם רוצים להפעיל צרכן "זולל", כמו מנוע למשל, אין להפעיל אותו ישירות מפינים אלו אלא יש להשתמש בממסר, טרנזיסטור או דרייבר. מתח הכניסה של הארדואינו נע בין 7V ל-12V, אך בחיבור לבטריה חיצונית מומלץ לחבר 9V. אפשרות אחרת לאספקת מתח היא באמצעות כבל USB, אז הארדואינו מקבל מתח מהמחשב. שפת התכנות של הארדואינו היא שפת C, היא שפה נוחה לשימוש וקיימות בה ספריות ספציפיות שמקלות על כתיבת הקוד. הארדואינו משמש בפרויקט החממה, כמיקרו-בקר הראשי השולט על כל רכיבי החומרה האחרים.

3.5.2 ספק כוח

המכשיר שבאמצעותו אנו מספקים מתח לכלל המערכות בחממה אשר צורכות מתח הפעלה של 12V ובזרמים גבוהים מהזרם שיציאות הארדואינו מסוגלות לספק (40mA). המתח שהוא מוציא הוא 12V והזרם המקסימאלי הוא 4A. בחרנו דווקא בספק כוח של 12V מכיוון שזהו מתח העבודה של רוב הרכיבים בחממה שלנו כגון מנועים, מאווררים וברז חשמלי.

3.5.3 מאוורר

מאוורר של מעבד מחשב. מתח העבודה שלו הוא 12V. המאוורר מקרר את החממה ע"י יניקת החום מתוכה החוצה לפי העקרון המופיע בתרשים הבא.



תרשים 13 : מאוורר יונק חום מהחממה

בחרנו דווקא במאוורר זה, כיוון שמשקלו קל יחסית, ולכן ניתן להתקין אותו בקלות על החומר ממנו בנויה החממה. עם זאת המאוורר לא מאוד אפקטיבי בפני עצמו להורדת טמפרטורה של אזור רחב יותר מאשר מעבד של מחשב. בחממה אנו משתמשים בארבעה מאווררים.

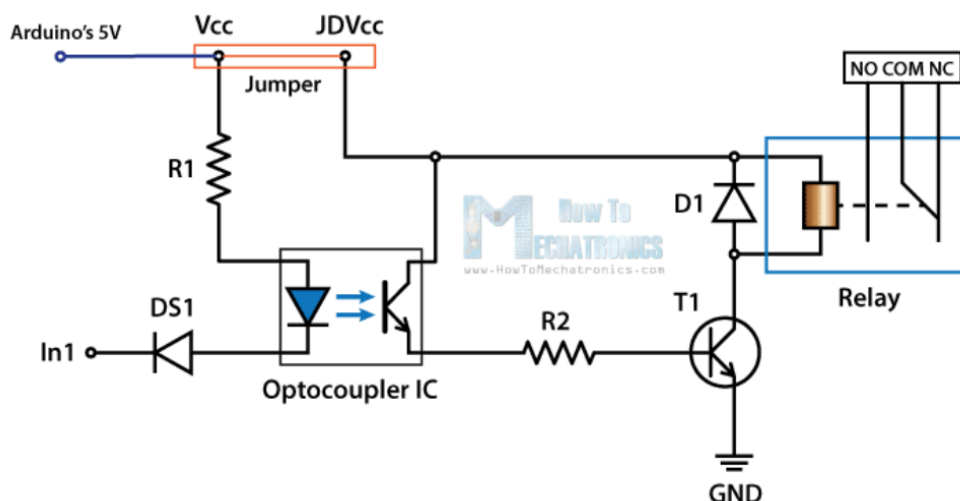


תרשים 14 : תמונת המאוורר שבו השתמשנו

3.5.4 ממסר

ממסר הוא התקן המכיל מפסק וסליל. כאשר זרם זרם בסליל, הסליל יוצר שדה מגנטי, הגורם לתנועה המועברת למפסק, והמפסק עובר ממצב "רגיל" למצב "מופעל". יש הפרדה "גלונית" (בידוד מושלם) בין חיבורי הסליל לחיבורי המפסק, כך שהתקן זה מאפשר למשל חיבור וניתוק של מעגלי זרם גבוה על ידי מעגלי זרם נמוך, תוך שמירת הבידוד ביניהם.

בחרנו ממסר שמתח ההפעלה של הסליל הוא 5VDC וזרם הפעלת הסליל הוא 5mA ולכן ניתן לחברו ליציאות דיגיטליות של הארדואינו. ניתן לחבר לממסר צרכן של עד 30VDC/10A.

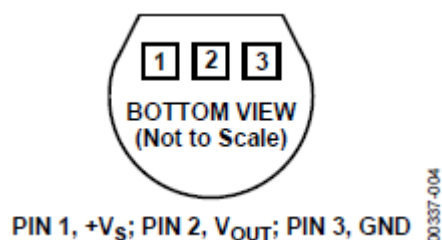


תרשים 15: סכמה חשמלית של ממסר

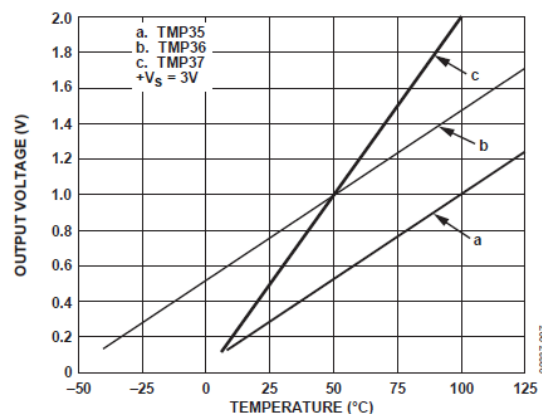
כדי להפעיל את הממסר צריך לספק לכניסת IN שלו 0 לוגי. יש אפשרות לנתק את ה- Vcc של הארדואינו ממתח הפעלת הסליל, על מנת לנטרל רעשי מיתוג למתח של הארדואינו, אופציה זו לא נבחרה בפרויקט, כדי לא להוסיף ספק כח נוסף למערכת.

3.5.5 חיישן טמפרטורה

TMP36 הוא חיישן טמפ' למתח נמוך אשר במוצאו המתח הוא פרופורציונאלי לטמפ' בצלזיוס. הוא לא דורש כיוול חיצוני בכדי לספק דיוק של $\pm 1^{\circ}\text{C}$ ב- 25°C ודיוק של $\pm 2^{\circ}\text{C}$ בכל טווח הטמפרטורות שלו (-40°C עד $+125^{\circ}\text{C}$). מתח ההפעלה שלו V_s הוא 5V.



תרשים 16: תיאור פינים של חיישן הטמפרטורה



תרשים 17: גרף מתח כפונקציה של הטמפרטורה

TMP36 מתנהג לפי גרף b. ונוסחת מיפוי מתח המוצא בוולטים שלו לטמפרטורה בצלזיוס היא:

$$T = 100 * V - 50$$

3.5.6 XBee S1 802.15.4 RF Module

זהו מודול מאוד פופולרי לתקשורת אל-חוטית בתדר 2.4GHz. מחברת Digi. מודול זה מממש פרוטוקול 802.15.4 המהווה בסיס לפרוטוקול Zigbee, ומספק דרך קלה לתקשר בעזרת פקודות טוריות. מודולי XBee מאפשרים ליצור קשר יציב בין בקרים, מחשבים, חיישנים, מערכות או כל דבר אחר התומך בתקשורת טורית. זהו מודול מסדרה 1 (S1) המיועדת ליצירת קשר נקודה לנקודה. מודולים מסדרה 1 מגיעים עם הגדרות מפעל כך שהם יתחברו אחד עם השני ישיר לאחר ההפעלה, כך שאפשר לראות בהם תחליף ישיר לכבל טורי. אם ישנם כמה מודולים בתווך שידור, כל מה שישלח לרגל Rx של מודול אחד יופיע ברגל Tx של כל המודולים האחרים המחוברים אליו. יש אפשרות לתכנת את המודול בעזרת פקודות טוריות להתנהג בצורה אחרת, כדי לשלוח נתונים למודול ספציפי וגם לדעת מאיזה מודול מגיע המידע.

בנוסף לקווי Rx ו Tx-למודול יש 6 כניסות אנלוגיות (ADC) עם רזולוציה של 10 ביט ו-8 קווי IO דיגיטליים המאפשרים חיבור ישיר של חיישנים או מקור מידע אחר שהנתונים שלו יכולים להיות משודרים בין המודולים. מודול זה מגיע עם אנטנת חוט בעלת ביצועים קצת יותר טובים מאנטנת שבב, אבל החוט אמור לעמוד זקוף ודורש מקום לכך. מאפיינים:

- מתח עבודה: 2.8-3.8v
- זרם: 250mA @3.3v
- עוצמת שידור: 60mW
- קצב שידור: 250kbps
- מרחק מקסימלי: 1500 מטר (תלוי בסוג המודול וסוג האנטנה)

- מספר קווי DIO : 8
- מספר קווי ADC : 6
- רזולוציית ADC : 10bit

בפרויקט השתמשנו בשני מודולי XBee : אחד חובר לארדואינו והוגדר כ- Coordinator והשני חובר לטנסיומטר של הצמח והוגדר כ- End Device.

בחרנו במודול ה-XBee בתור משדר/מקלט, מכיוון שהוא צורך הספק נמוך, קצב השידור שלו נמוך אך מתאים לצרכים של הפרויקט שלנו (קצב נתונים נמוך), הגדרתו פשוטה והוא מתאים למרחקים גדולים בשטח פתוח. על מנת להגדיר את ה-XBee צריך לשנות רגיסטרים פנימיים שלו.

רגיסטרים שבהם השתמשנו בפרויקט :

DL – 32 הביטים הנמוכים של כתובת יעד, את 32 הביטים הגבוהים השארנו 0x0.

MY – 16 ביט של כתובת המקור.

ID – מספר המייצג את ה-PAN שהוא מספר זהה לכל ה-XBee שנמצאים באותה רשת לוגית, במידה ויש באותו תווך רשתות שונות, לכל רשת מקצים PAN ID שונה.

CE – בוחר אם ה-XBee משמש כ- Coordinator או כ- End Device. ה- Coordinator אחראי על סינכרון השידורים ברשת.

AP – מגדיר צורת תקשורת טורית בין ה-XBee לבקר מארח. אנו בחרנו צורת תקשורת API שמפורטת בהמשך.

EE – אפשרור הצפנה של השידורים האלחוטיים, על מנת למנוע חדירה לרשת ושינוי התנהגות המערכת

KY – מפתח AES עבור ההצפנה.

D0 – קונפיגורציה לפין DIO0/AD0

D1 – קונפיגורציה לפין DIO1/AD1

IR – קצב דגימה ב-mSec של הכניסות.

ST – זמן ב-mSec לפני כניסת הרכיב למצב שינה sleep.

SP – זמן ב-10x mSec של משך השינה

XBee Arduino Shield 3.5.6.1

בצד הארדואינו השתמשנו בכרטיסון XBee Arduino Shield אשר ממיר את מתח הארדואינו מ-5V ל-3.3V של ה-

XBee וממיר את מתחי ה-RX/TX טוריים של הארדואינו למתחים מתאימים בכניסות הטוריות של ה-XBee.

את הרגיסטרים של ה-XBee בצד של הארדואינו שינינו באמצעות פקודות API בערוץ הטורי בין ה-XBee לארדואינו. פורמט פקודת ה-API מתואר להלן :

Frame fields	Byte	Description
Start delimiter	1	0x7E
Length	2 - 3	Most Significant Byte, Least Significant Byte
Frame data	4 - n	API-specific structure
Checksum	n + 1	1 byte

תרשים 18 : פורמט הודעת API כללית

על מנת לשנות רגיסטרים השתמשנו בפקודת API מסוג AT Command Frame. פורמט Frame Data של פקודה זו הוא :

Frame data fields	Offset	Description
Frame type	4	0x08
Frame ID	5	Identifies the UART data frame for the host to correlate with a subsequent ACK (0x88). If set to 0 , the device does not send a response.
AT command	6-7	Command name: two ASCII characters that identify the AT command.
Parameter value	8-n	If present, indicates the requested parameter value to set the given register. If no characters are present, it queries the register.

תרשים 19: פורמט API של פקודת AT Command Frame

ערך AT Command בבתים 6,7 הם ערכי ASCII של פקודת AT (לכל רגיסטר 2 תווים) והפרמטרים הינם הערכים שרוצים לעדכן ברגיסטר.

טבלה 3: ערכי רגיסטרים ב-XBee בצד של הארדוואינו

רגיסטר	ערך	משמעות
FR	ללא ערך	מאתחל את כל הרגיסטרים לערכי הגדרות יצרן
DL	0x5678	כתובת יעד (כתובת מקור של XBee צמח)
MY	0x1234	כתובת מקור
ID	0x3188	חייב להיות זהה בשני ה-XBee
CE	0x1	Coordinator
AP	0x1	מאפשר פורמט API בין XBee לבקר מארח
EE	0x1	אפשר הצפנה
KY	0x526f666f724c616e6433313838373932	מפתח הצפנה AES
WR	ללא ערך	שמירת ערכי הרגיסטרים ב-XBee (נשמרים לאחר כיבוי מתח)

3.5.6.2 XBee Module

בצד הצמח השתמשנו במודול בסיסי של ה-XBee. את הרגיסטרים של ה-XBee בצד של הצמח שינינו באמצעות תוכנת XCTU של Digi. לאחר שינוי הרגיסטרים, הערכים החדשים נשמרים בזיכרון של ה-XBee, גם לאחר כיבוי והפעלה.



תרשים 20: מסך שינוי רגיסטרים בתוכנת XCTU

טבלה 4: ערכי רגיסטרים ב- XBee בצד של הצמח

רגיסטר	ערך	משמעות
DL	0x1234	כתובת היעד (כתובת המקור של ה-XBee בצד של הארדואינו)
MY	0x5678	כתובת המקור
ID	0x3188	חייב להיות זהה בשני ה-XBee
CE	0x0	End Device
EE	0x1	אפשר הצפנה
KY	0x526f666f724c616e6433313838373932	מפתח הצפנה AES
D0	0x2	הגדרה של פין DIO0/AD0 כ- ADC
D1	0x2	הגדרה של פין DIO1/AD1 כ- ADC
IR	0x1f4	דגימה כל 500mSec של כל הכניסות שמאופשרות, במקרה שלנו AD0, AD1. ושידור הודעת RX Packet IO ל- XBee בצד ארדואינו
ST	0x1f4	המתנה של 500mSec לפני כניסה לשינה
SP	0x190	שינה למשך 4000mSec.

משמעות הפרמטרים ST, IR, SP היא מחזור העבודה של ה-XBee בצד של הצמח. IR זה קצב דגימת המידע, ST זה כמה זמן עד שה-XBee יכנס למצב שינה לאחר שסיים את כל הפעולות שלו ו-SP זה כמה זמן ה-XBee יהיה במצב שינה. כלומר לפי ההגדרות שלנו כל 5 שניות ישלח נתון ADC ואח"כ ה-XBee יכנס למצב שינה. חשוב להכניס את ה-XBee בצד של הצמח למצב שינה, על מנת לחסוך בסוללה, כיוון שהמעגל הוא מרוחק ורצוי לא להחליף סוללה לעיתים קרובות. בחרנו משך שינה של 4 שניות בכל מחזור של 5 שניות (80%) על מנת להדגים שינויים

בצורה מהירה, אך במערכת אמיתית ניתן להגדיר מחזור שינה ארוך יותר למשל כל 10 דקות, כיוון שהשינויים בטנסיומטר הם איטיים.

טבלה 5: פינים בשימוש ב-XBee צמח

פין	שם	תפקיד
1	VCC	מתח אספקה 3.3v
10	GND	אדמה
14	VREF	מתח ייחוס ל-ADC
19	AD1	ADC1
20	AD0	ADC0

ה-XBee בצד של הצמח שולח הודעת IO כל 5 שניות, XBee ארדואינו מפענח הודעה זו ושולח לארדואינו הודעת RX Packet IO בפורמט הבא:

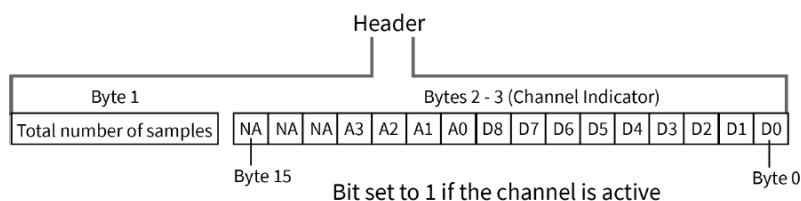
Frame data fields	Offset	Description
Frame type	4	0x81 0x83
Source address	5-6	MSB first, LSB last.
RSSI	7	Received Signal Strength Indicator. The Hexadecimal equivalent of (-dBm) value. For example if RX signal strength is -40 dBm, then 0x28 (40 decimal) is returned.
Options	8	Bit field: bit 0: Reserved bit 1: Address broadcast bit 2: PAN broadcast bits 3-7: Reserved
RF data	9-n	Up to 100 bytes per packet.

תרשים 21: הודעת RX Packet IO (16bit)

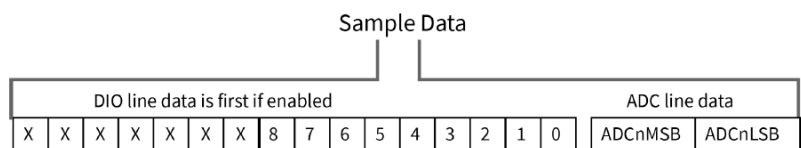
פורמט RF Data בתרשים הבא:

I/O data format

I/O data begins with a header. The first byte of the header defines the number of samples forthcoming. The last two bytes of the header (Channel Indicator) define which inputs are active. Each bit represents either a DIO line or ADC channel. The following figure illustrates the bits in the header.



Sample data follows the header and the channel indicator frame determines how to read the sample data. If any of the DIO lines are enabled, the first two bytes are the DIO sample. The ADC data follows. ADC channel data is represented as an unsigned 10-bit value right-justified on a 16-bit boundary. The following figure illustrates the sample data bits.



תרשים 22: פורמט RF Data בהודעת IO

הארדוואינו מפענח הודעה זו ומוציא ממנה את ערכי AD0, AD1. 2 בתים לכל נתון.

3.5.7 ברז חשמלי

השתמשנו בברז חשמלי מסוג DC Latch. על ידי פולס קצר של מתח 9-20V הברז נפתח ועל ידי פולס קצר של מתח 9-20V בקוטביות הפוכה הברז ייסגר. רוחב הפולס צריך להיות לפחות 100mSec. היתרון בסוג כזה של ברז שהוא אינו מבובז חשמל ואין צורך לספק מתח במשך זמן ההשקייה שיכול להיות מס' דקות.



תרשים 23: ברז חשמלי

3.5.8 סוללת 9V

הסוללה משמשת לאספקת מתח למעגל שנמצא ליד הצמח. מעגל זה מרוחק ממקור מתח ולכן חיברנו אותו לסוללת 9V. לסוללה של 9v יש אנרגיה של 600mAh.

מעגל הצמח צורך כ- 1mA.

ה-XBee צורך 45mA בזמן שידור ו- 200μA בזמן שאינו משדר, ופחות מ- 10μA במצב שינה, לפי ערכים אלו חישוב חישבו מקורב שמעגל הצמח צורך כ- 1mA לפי מחזור שינה של 4 שניות שינה כל 5 שניות. זרם זה יספיק ל-600 שעות של סוללה (כחודש עבודה). כדי להגדיל זמן זה במערכת אמיתית, ניתן להגדיל את משך הזמן במצב שינה בצורה משמעותית, לדוגמה על כל שניה של דגימה, מצב שינה של דקה.

כדי לתת אינדיקציה למשתמש שהסוללה עומדת להיגמר ויש להחליפה, דגמנו גם את מתח הסוללה, כאשר הנחנו ש 7.5V הוא מצב ריק של הסוללה.

3.5.9 מייצב מתח מעגל Pololu S7V8F3

במעגל הצמח השתמשנו במייצב מתח מבוסס רכיב ממיר מתח TPS63060. המעגל מקבל את מתח הסוללה 9v ומספק 3.3v ל- XBee.

מתחי העבודה של המעגל הם 2.7-11.8v והוא מספק 3.3v מיוצב וזרם יציאה מקסימאלי של 1A.

3.5.10 טנסיומטר

טנסיומטר הוא חיישן להערכת זמינות המים לצמח באמצעות מדידת מתח המים בקרקע. החיישן מדמה את פעולת השורשים של הצמח כאשר הקרקע יבשה כמות זעירה של מים יוצאת מהטנסיומטר לאדמה ונוצר לחץ בטנסיומטר, וכאשר האדמה לחה מים חוזרים לטנסיומטר והלחץ יורד.

מתח האספקה למד הלחץ של הטנסיומטר הוא 3-12v. ויציאת מד הלחץ היא 0-2v כאשר 0v מייצג לחץ 0 או אדמה לחה ולחץ 2v מייצג לחץ של 1000mBar, כלומר אדמה יבשה לגמרי.



תרשים 24: טנסיומטר אנלוגי

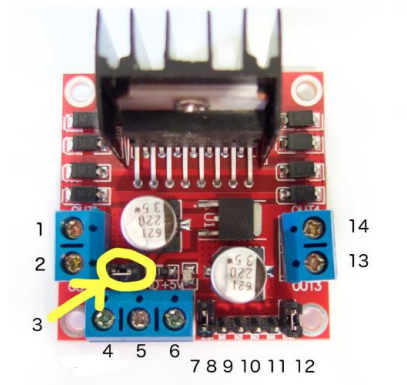
המכשיר מסייע לחקלאי להחליט מתי וכמה להשקות. במערכת שלנו נתון הלחץ משודר ע"י ה- XBee למעגל הבקרה וההחלטה אם להשקות או לוקחת בחשבון את יובש האדמה, כדי לא לבזבז מים כאשר האדמה לחה וצמח אינו זקוק למים.

3.5.11 לוח מקשים 4x4

לוח המקשים הינו מטריצת לחצנים אשר מכילה 16 לחצנים ביניהם 10 ספרות, כוכבית, סולמית ו-4 אותיות באנגלית. באמצעות לוח המקשים ניתן להקיש קלט מתבקש לתוכנה בארדואינו. כאשר נלחץ מקש נוצר קצר בין פין Row בודד לפין Col בודד, פונקציית הספרייה של הארדואינו מספקת '1' לכל אחד מהפינים של ה- Row לסירוגין ובודקת באיזה פין של Col יש '1' אם לא נמצא אף פין Col שיש בו '1' הפונקציה עוברת ל- Row הבא אם אף אחד מה- Row לא מפעיל Col אז לא נלחץ שום לחצן.

3.5.12 דרייבר מנוע HW-95

דרייבר מנוע מבוסס רכיב L298N משמש לבקרה של 2 מנועי DC. דרייבר הוא מעגל שבו משתמשים בשביל לשלוט במעגל אחר. בדרך כלל משתמשים בהם לוויסות זרם במעגל.



תרשים 25 : דרייבר מנוע

בטבלה הבאה הפינים שנמצאים בשימוש בפרויקט שלנו :

טבלה 6 : פינים בדרייבר בשימוש בפרויקט

מס' פין	שימוש
1	מתח $V+$ למנוע 1
2	מתח $V-$ למנוע 1
4	אספקת מתח $12V$
5	אדמה
7	Enable למנוע 1
8	IN1
9	IN2
10	IN3
11	IN4
12	Enable למנוע 2
13	מתח $V+$ למנוע 2
14	מתח $V-$ למנוע 2

3.5.13 כונן דיסקים

מכשיר שבאמצעותו מסוגל המחשב לקרוא מידע מתוך דיסקים. בפרויקט שלנו מנגנון הפתיחה והסגירה של הכונן משמש בתור דלת הכניסה לחממה. הפעלת המנוע נעשית באמצעות דרייבר מנוע.

LDR 3.5.14

נקרא גם פוטורזיסטור. זהו רכיב חשמלי אשר משנה את התנגדותו בהתאם לרמת האור אליו הוא חשוף כתוצאה מפגיעת פוטונים.

ל-LDR יש התנגדות של 400Ω כאשר עוצמת התאורה היא מקסימאלית 1000LUX , $9K\Omega$ בעוצמת תאורה נמוכה של 10LUX והתנגדות של $1M\Omega$ בחושך מוחלט. הזרם המקסימאלי שמותר להעביר דרך ה-LDR הוא 75mA .

סרט לד 3.5.15

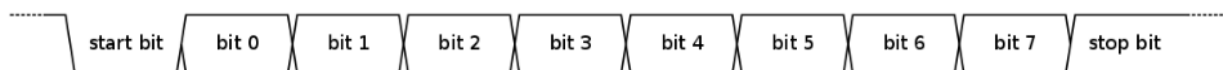
לד הוא דיודה אשר פולטת אור כאשר עובר בה זרם. סרט הלדים מו מתח ההפעלה לסרט הלדים הוא 12V .

מנוע תריס 3.5.16

מנוע התריס הוא מנוע DC המופעל במתח 12V . הוא מסוגל להסתובב ימינה או שמאלה בהתאם לקוטביות המתח על הכניסות שלו. הוא מחובר באמצעות דרייבר לארדואינו. על מנת להאט את קצב הסיבוב של המנוע השתמשנו ב-PWM ברגל Enable של הדרייבר.

ממשק טורי UART 3.5.17

UART הוא פרוטוקול תקשורת טורית בין 2 רכיבים. התקשורת היא full duplex כיוון שלכל כיוון תקשורת יש קו נפרד TX, RX. בכל פעם מועברים 8bit של מידע לפי התרשים הבא



תרשים 26: פורמט UART

השתמשנו בממשק טורי בפרויקט בשני מקומות:

- ערוץ טורי 0 של הארדואינו חובר למחשב באמצעות USB ושימש לתקשורת בין הארדואינו ל-GreenSee
- ערוץ טורי 1 של הארדואינו חובר ל-XBee Arduino Shield ושימש לתקשורת בין הארדואינו ל-XBee Coordinator.

מאפייני התקשורת שהוגדרו לשני הערוצים הם:

Baud Rate – 9600

Data Bits – 8

Stop Bits – 1

Parity – None (ביט בדיקת זוגיות לא בשימוש)

Flow Control – None (לא היה צורך כיוון שקצב המידע שהועבר נמוך וזמן העיבוד מהיר)

4 מדידות**4.1 מדידות טנסיומטר****מחוץ לאדמה**

כשאינן מים : 10.5mV

כשיש מים : 226mV

בתוך האדמה

שמנו אדמה לחה מהגינה ומדדנו את מתח הטנסיומטר למשך מס' ימים :

טבלה 7 : מדידות מתח טנסיומטר

תאריך ושעה	מתח מהטנסיומטר [V]
13: 55 1.2.19	0.226
12: 25 2.2.19	0.182
17: 53 3.2.19	0.135
14: 24 4.2.19	0.155
21: 27 5.2.19	0.340
14: 04 6.2.19	0.615
20: 29 7.2.19	0.814
13: 23 8.2.19	0.812

לאחר השקיית האדמה מדדנו מתח 0.12V בטנסיומטר.

לאור המדידות קבענו מתח סף של 0.7V שמעליו החשבנו את האדמה כיבשה, ושהצמח צריך השקיה, ומתחת למתח זה הצמח אינו זקוק להשקיה.

פרמטר זה הינו ניתן לקונפיגורציה באמצעות תכנת GreenSee.

4.2 חישוב מתח סוללה ו-% שנותר

בהתאם לסעיף 3.5.8 תוכנת GreenSee מקבלת את מתח הסוללה במעגל הצמח ומבצעת חישוב אחוז סוללה בהתאם לחישוב הבא :

$$BatteryLevel \% = \frac{BatteryVoltage - 7.5}{1.5} * 100$$

4.3 חישוב טיימר 1sec

שעון הארדואינו הוא מתנד גבישי בעל תדר של 16MHz.

על מנת לקבל פסיקה פעם בשנייה, השתמשנו ב- prescale של 256 כלומר כל 256 פעימות של שעון הארדואינו המונה החומרתי של טיימר 3, TCNT3 עולה באחד, וכשהמונה עובר מ- 0xFFFF ל 0x0 מתרחשת פסיקה. לכן כדי להגיע לשנייה המונה צריך להיות מאותחל לערך הבא :

$$TCNT3 = 2^{16} - \frac{16e6}{256} = 3036$$

4.3.1 מדידת חיישן טמפרטורה

מד טמפרטורה מעבדתי הציג טמפרטורה של 22.2°C וחיישן הטמפרטורה שלנו לאחר המרות הציג $21-22^{\circ}\text{C}$.

4.3.2 מדידות RSSI

במרחק של 40m (בשטח לא לגמרי פתוח) ה-XBee צמח לא נקלט יותר וה-RSSI האחרון שנצפה היה -85dBm.

4.3.3 מדידת הזרם הנצרך מספק הכוח של 12V

ממדנו צריכת זרם של כל מערכת בנפרד.

טבלה 8: זרם נצרך במערכות

מצב נמדד	מדידת זרם
לדים דולקים	0.56A
מאווררים דולקים	0.34A
תריס יורד	0.11A
תריס עולה	0.15A
ברז חשמלי	2A (ערך זה לא נמדד, אלא חושב לפי התנגדות הסליל שהיא 6Ω)
דלת כניסה	1.12A
סה"כ מקסימום זרם עם כל המערכות עובדות ביחד	4.17A

ספק הכוח של מעגל ה-12V הוא של 4A ומספיק להפעיל את כל המערכות. למרות שכל המערכות ביחד צורכות 4.17A, הסיכוי שכולם יעבדו יחד הוא אפסי, בשל העובדה שהתריס, הברז החשמלי והדלת מופעלים לפרקי זמן מאוד קצרים.

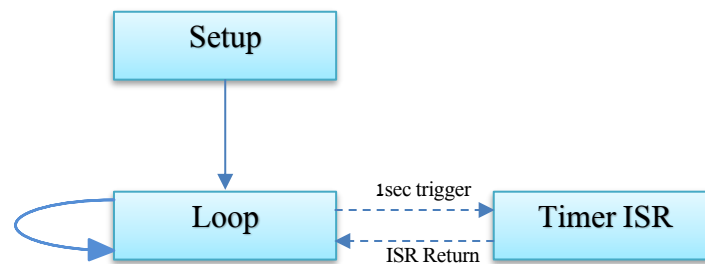
5 תיאור תכנה

5.1 תכנת GreenDo

תכנת GreenDo כתובה בשפת C במעבד של ארדואינו.

התכנה בנויה מ-3 חלקים עיקריים:

- (1) פונקציית Setup שבה מאותחלים כל רכיבי החומרה והמשתנים.
- (2) פונקציית loop שרצה בלולאה אינסופית
- (3) פסיקת טיימר שמופעלת אחת לשנייה



תרשים 27: תרשים זרימה כללי GreenDo

כל הפונקציות הכתובות בתוכנה זו הינן פונקציות שכתבנו מאפס ולא פונקציות ספרייה.

5.1.1 פונקציות לטיפול בשעון זמן

פונקציות אלו מנהלות את השעון היומי.

5.1.1.1 SendLocalTime

שולחת את השעון היומי לתצוגה בתוכנת GreenSee.

5.1.1.2 HandleDayClock

פונקציה זו מעדכנת את היום, השעה, הדקה והשנייה בשעון היומי. פונקציה זו נקראת פעם בשנייה.

5.1.2 פונקציות כיבוי והפעלת מערכות בחממה

פונקציות אלו מפעילות ומכבות מערכות בחממה, ע"י שינוי ערכי פינים בארדואינו.

5.1.2.1 WaterValveDriverDisable

פונקציה זו מבטלת את פעולת הדרייבר שמפעיל את ברז המים. זוהי פונקציית callback שנקראת ע"י הטיימר התוכנתי כאשר הוא פוקע.

5.1.2.2 Open Water Valve

פונקציה זו נותנת פקודה לדרייבר שיפתח את ברז המים. הפולס מהדרייבר לברז המים נמשך שנייה, מכיוון שהברז הוא מסוג Latch, דבר זה אומר שהברז נשאר במצב האחרון שהוגדר.

CloseWaterValve 5.1.2.3

פונקציה זו נותנת פקודה לדרייבר שיסגור את ברז המים. הפולס מהדרייבר לברז המים נמשך שנייה.

CloseWaterValveSetup 5.1.2.4

פונקציה זו מבצעת את אותו הדבר כמו הפונקציה *CloseWaterValve*, אך לעומתה יש בה שימוש בפונקציית *Delay*, מכיוון שקוראים לה מפונקציית ה-*setup*, שבה הטיימרים עוד לא הוגדרו.

DoorDriverDisable 5.1.2.5

פונקציה זו מבטלת את פעולת הדרייבר שפותח/סוגר את דלת הכניסה. זוהי פונקציית *callback* שנקראת ע"י הטיימר התוכנתי כאשר הוא פוקע.

OpenDoor 5.1.2.6

פונקציה זו נותנת פקודה לדרייבר שיפתח את דלת הכניסה. הפולס מהדרייבר לדלת נמשך שנייה. כמו כן הפונקציה מעדכנת את משתנה המצב *DoorIsClosed* ל-*false*.

CloseDoor 5.1.2.7

פונקציה זו נותנת פקודה לדרייבר שיסגור את דלת הכניסה. הפולס מהדרייבר לדלת נמשך שנייה. כמו כן הפונקציה מעדכנת את משתנה המצב *DoorIsClosed* ל-*true*.

CloseDoorSetup 5.1.2.8

פונקציה זו מבצעת את אותו הדבר כמו הפונקציה *CloseDoor*, אך לעומתה יש בה שימוש בפונקציית *Delay*, מכיוון שקוראים לה מפונקציית ה-*setup*, שבה הטיימרים עוד לא הוגדרו.

FansOn 5.1.2.9

פונקציה זו נותנת פקודה לממסר שיפעיל את כל ארבעת המאווררים. כמו כן, הפונקציה בודקת את משתנה המצב *FanIsOn*, אם הוא במצב *true* היא ישר יוצאת מהפונקציה מבלי לבצע את הפקודות הכתובות בה, ואם הוא *false* הפונקציה משנה אותו ל-*true* ומבצעת את כל הפקודות בה.

FansOff 5.1.2.10

פונקציה זו נותנת פקודה לממסר שיכבה את כל ארבעת המאווררים. כמו כן, הפונקציה בודקת את משתנה המצב *FanIsOn*, אם הוא במצב *false* היא ישר יוצאת מהפונקציה מבלי לבצע את הפקודות הכתובות בה, ואם הוא *true* הפונקציה משנה אותו ל-*false* ומבצעת את כל הפקודות בה.

LightsOn 5.1.2.11

פונקציה זו נותנת פקודה לממסר שידליק את סרט הלדים. כמו כן, הפונקציה בודקת את משתנה המצב `LightIsOn`, אם הוא במצב `true` היא ישר יוצאת מהפונקציה מבלי לבצע את הפקודות הכתובות בה, ואם הוא `false` הפונקציה משנה אותו ל-`true` ומבצעת את כל הפקודות בה.

LightsOff 5.1.2.12

פונקציה זו נותנת פקודה לממסר שיכבה את סרט הלדים. כמו כן, הפונקציה בודקת את משתנה המצב `LightIsOn`, אם הוא במצב `false` היא ישר יוצאת מהפונקציה מבלי לבצע את הפקודות הכתובות בה, ואם הוא `true` הפונקציה משנה אותו ל-`false` ומבצעת את כל הפקודות בה.

ShutterDriverDisable 5.1.2.13

פונקציה זו מבטלת את פעולת הדרייבר ששולט על התריס. זוהי פונקציית `callback` שנקראת ע"י הטיימר התוכניתי כאשר הוא פוקע.

ShutterUp 5.1.2.14

פונקציה זו נותנת פקודה לדרייבר שיעלה את התריס. הפולס מהדרייבר לתריס נמשך לפרק הזמן שנקבע ע"י המשתמש בתוכנה `GreenSee`. כמו כן הפונקציה בודקת את משתנה המצב `ShutterIsUp`, אם הוא במצב `true` היא ישר יוצאת מהפונקציה מבלי לבצע את הפקודות הכתובות בה, ואם הוא `false` הפונקציה משנה אותו ל-`true` ומבצעת את כל הפקודות בה.

ShutterDown 5.1.2.15

פונקציה זו נותנת פקודה לדרייבר שיוריד את התריס. הפולס מהדרייבר לתריס נמשך לפרק הזמן שנקבע ע"י המשתמש בתוכנה `GreenSee`. כמו כן הפונקציה בודקת את משתנה המצב `ShutterIsUp`, אם הוא במצב `false` היא ישר יוצאת מהפונקציה מבלי לבצע את הפקודות הכתובות בה, ואם הוא `true` הפונקציה משנה אותו ל-`false` ומבצעת את כל הפקודות בה.

5.1.3 פונקציות טיימרים תוכנתיים

טיימרים תוכנתיים אשר נקראים ומנוהלים מתוך שגרת הטיימר (פעם בשנייה). הטיימרים הם מסוג `One Shot`, כלומר כשהם פוקעים הם קוראים לפונקציית `callback` ומבוטלים עד לדריכה הבאה.

SetSoftTimer 5.1.3.1

פונקציה זו מקבלת את שם הטיימר, את משך הזמן שלו ומצביע לפונקציית ה-`callback`. הפונקציה דורכת את הטיימר המתאים למשך הזמן הרצוי. הערה: משך הזמן שהפונקציה מקבלת הוא כמות השניות הרצויה ועוד אחד.

DisableSoftTimer 5.1.3.2

פונקציה זו מקבלת את שם הטיימר ומבטלת אותו.

5.1.3.3 CheckSoftTimers

פונקציה זו סורקת את כל הטיימרים התוכנתיים. בטיימרים שפועלים מעדכנת את משך הזמן שנותר עד לפקיעת הטיימר, ובמידה והטיימר פוקע, קוראת לפונקציית ה-callback ומבטלת את הטיימר.

5.1.4 פונקציות ממשק בין תוכנת GreenSee ל-GreenDo

ממשק בין GreenSee ל-GreenDo מיושם בערוץ הטורי של הארדואינו והמחשב באמצעות פלאג ה-USB של כרטיס האדואינו.

כדי לממש תקשורת זו יצרנו פרוטוקול פשוט המבוסס על מחרוזות בפורמט הבא :

טבלה 9 : פרוטוקול תקשורת בין תכנת GreenSee ל-GreenDo

מס' תו בהודעה	תיאור
1	תו התחלת הודעה - ~
2	תו אות בודד המסמן את סוג ההודעה
3	תו =
4-n	מחרוזת ערכים מספריים שמשמשים כפרמטר אחד או יותר בהודעה. אם יש יותר מפרמטר אחד, הפרמטרים מופרדים ע"י פסיק
n+1	תו linefeed (10 in decimal)

הטבלה הבאה מפרטת את סוגי ההודעות, אשר נשלחות מ-GreenDo ל-GreenSee :

טבלה 10 : הודעות מ-GreenDo ל-GreenSee

תו פקודה	סוג פרמטר	דוגמה
D	מחרוזת להדפסה לחלון debug בתוכנת GreenSee	~D=Hello Greenhouse
C	מחרוזת שבה עם טמפרטורה עכשווית שנמדדה	~C=26
T	מחרוזת לתצוגת הזמן הנוכחי שבארדואינו	~T=20:25
B	מחרוזת שבה עם מתח הסוללה הנמדד של מעגל הצמח	~B=8.3
M	מחרוזת עם מתח הנמדד של הטנסיומטר	~M=0.71
L	מחרוזת עם רמת עוצמת האור בחממה	~L=638
R	מחרוזת עם עוצמת הקליטה (RSSI)	~R=43

הטבלה הבאה מפרטת את סוגי ההודעות אשר נשלחות מ-GreenSee ל-GreenDo :

טבלה 11 : הודעות מ-GreenSee ל-GreenDo

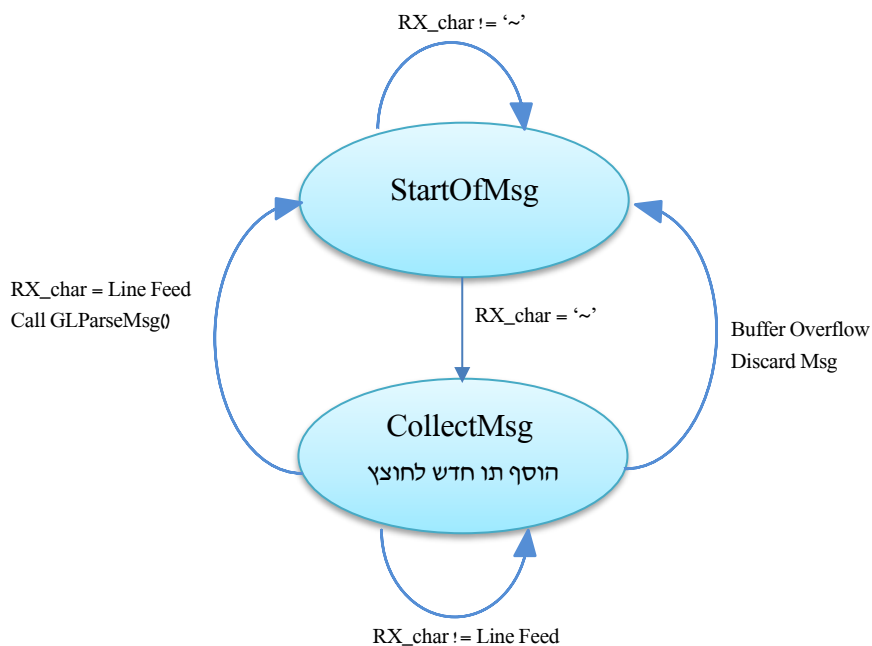
תו פקודה	סוג פרמטר	דוגמה
I	מחרוזת שבה נתונה תוכנית ההשקייה והתאורה לכל יום בשבוע: יום בשבוע (לפי מיקום במערך), אפשר השקייה לאותו היום, שעת ההשקייה מהמשתמש בשעות ודקות, משך זמן ההשקייה בשניות, שעת תחילת התאורה בשעות ודקות ושעת סיום התאורה בשעות ודקות	$\sim I=0, 1, 10, 15, 600, 19, 55, 3, 47$ יום ראשון, אפשר השקייה בשעה 10: 15 למשך 10 דקות (600 שניות) וזמן תאורה בין 19: 55 ל-47: 3
L	מחרוזת שבה נתונה הטמפרטורה שמוגדרת כנמוכה ע"י המשתמש	$\sim L=20$
H	מחרוזת שבה נתונה הטמפרטורה שמוגדרת כגבוהה ע"י המשתמש	$\sim H=35$
U	מחרוזת שבה נתונה סף רמת יובש האדמה	$\sim U=0.72$
V	מחרוזת שבה נתונה רמת עוצמת האור שנחשבת כנמוכה ע"י המשתמש	$\sim V=300$
Y	מחרוזת שבה נתון הזמן בשניות שהוגדר ע"י המשתמש להעלאת התריס	$\sim Y=7$
Z	מחרוזת שבה נתון הזמן בשניות שהוגדר ע"י המשתמש להורדת התריס	$\sim Z=6$
T	מחרוזת שבה נתונים היום בשבוע, השעה, הדקה והשנייה הנוכחיים לעדכון השעון ב-GreenDo	$\sim T=5, 8, 20, 32$ יום שישי 8: 20: 32
X	מחרוזת שבה מקבלים מספר שמייצג את הבדיקה שיש לעשות במערכות. 1 = הפעלת מאווררים 2 = כיבוי מאווררים 3 = הדלקת סרט לדים 4 = כיבוי סרט לדים 5 = העלאת תריס 6 = הורדת תריס 7 = פתיחת ברז מים 8 = סגירת ברז מים	$\sim X=3$

GreenSeeHandler 5.1.4.1

קליטת byte חדש מהערוץ הטורי וקריאה למכונת המצבים של הקליטה.

GLStateMachine 5.1.4.2

לשם קליטת הודעה מ-GreenSee התוכנה מיישמת את מכונת המצבים הבאה:



תרשים 28: מכונת מצבים קליטת הודעה מ-GreenSee

GLParseMsg 5.1.4.3

פונקציה המפענחת מחרוזת שנאספה במצב CollectMsg ומעדכנת את המשתנים הגלובאליים של התוכנה לפי המחרוזת שהתקבלה בהתאם לטבלה 11.

במידה והפרמטר הוא בודד, התוכנה ממירה אותו לערך מספרי ע"י פונקציית ToInt() או ToFloat(). במידה ויש מס' פרמטרים המופרדים ע"י תו פסיק התוכנה מאתרת את מיקום כל פרמטר במחזורת ואז עושה המרה של תת המחרוזת של הפרמטר לנתון מספרי.

פונקציות לתקשורת עם ה-XBee 5.1.5

בעזרת פונקציות אלו הארדוואינו מתקשר עם ה-XBee.

EndianSwap 5.1.5.1

פונקציה זו מחליפה את סדר הבתים של כל הודעה שנשלחת ל-XBee או מתקבלת מה-XBee כך שיהיו לפי פורמט ה-Endianness הנכון לכל צד. כלומר, הודעות המתקבלות מה-XBee שבהן ה-MSB נשלח קודם, יתהפכו על-ידי פונקציה זו כך שה-LSB ייקרא קודם, ולהפך כאשר נשלחות הודעות ל-XBee.

5.1.5.2 *SendAPIMsg*

פונקציה זו שולחת את הודעת ה-API שהוכנה ע"י הארדואינו אל ה-XBee. בנוסף, היא מחשבת את ה-Checksum של ההודעה.

5.1.5.3 *PrepareATCommand*

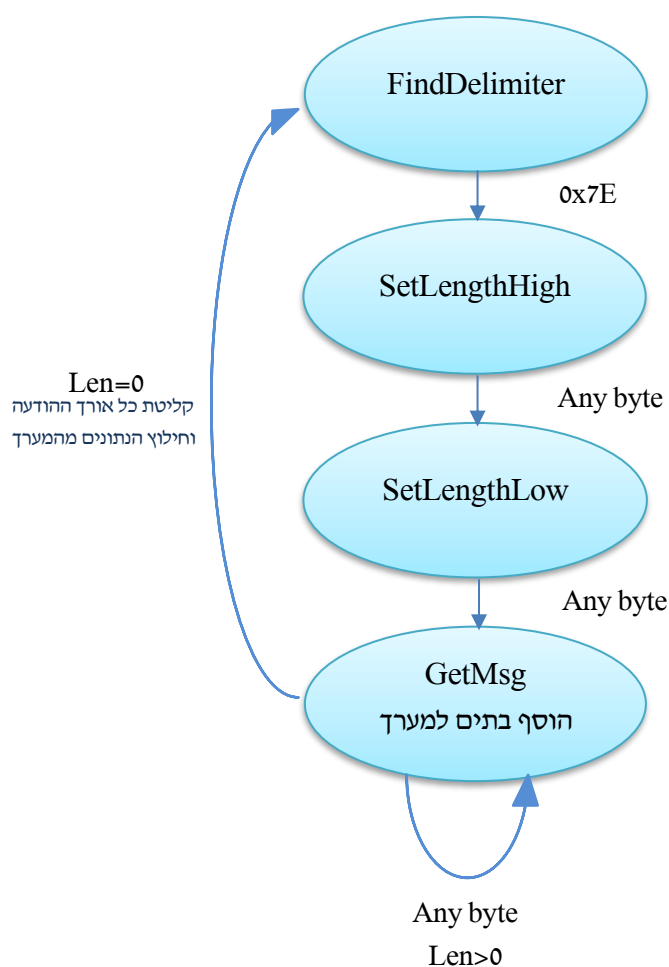
פונקציה זו מכינה את הודעת ה-API.

5.1.5.4 *ConfigureXBee*

פונקציה זו מגדירה את כל הרגיסטרים הנחוצים ב-XBee בצד של הארדואינו.

5.1.5.5 *XBeeHandler*

פונקציה זו עוזרת בקליטת הודעה מה-XBee, ובנוסף היא מפענחת את ההודעה ושולחת ל-GreenSee את הנתונים שהתקבלו: עוצמת הקליטה, המתח מהטנסיומטר ומתח הסוללה. XBee צמח שולח הודעה מסוג אחד לכך פונקציית XBeeHandler מניחה בפענוח ההודעה זו ההודעה שהתקבלה ומיקום הנתונים שלהן הוא במקום קבוע במערך הקליטה.



תרשים 29: מכוונת מצבים לקליטת הודעה

CheckIrrigationPlan 5.1.5.6

הפונקציה בודקת אם תוכנית ההשקייה של היום הנוכחי מאופשרת, אם הזמן הנוכחי (יום, שעה, דקה ושניה) הוגדר כזמן השקייה ואם המתח בטנסיומטר גבוה מהסף המופיע ב-GreenSee. היא קוראת לפונקציות ההשקייה ומפעילה טיימר למשך זמן ההשקייה אם כל התנאים הנ"ל מתקיימים.

פונקציות דלת כניסה 5.1.6

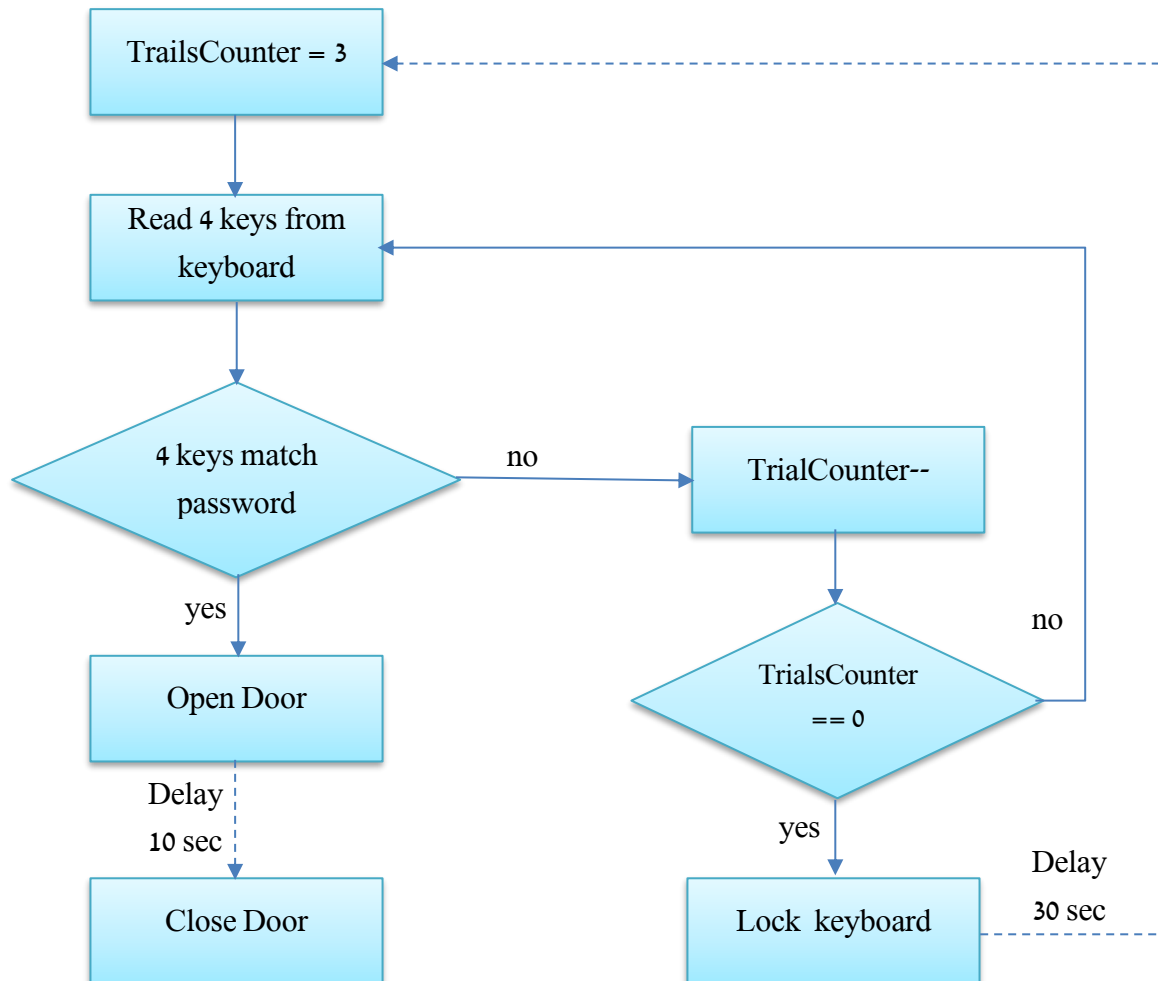
פונקציות לפתיחה וסגירה של דלת הכניסה לחממה.

KeyLockRelease 5.1.6.1

פונקציה זו משנה את משתנה המצב WrongKeyLock ל-false ומדפיסה הודעה למשתמש שהנעילה שוחררה ושניתן להקיש סיסמה שוב.

GreenHouseDoorHandler 5.1.6.2

פונקציה זו מבצעת את הלוגיקה של פתיחת הדלת באמצעות הקודן לפי תרשים הזרימה הבא :



תרשים 30 : תרשים זרימה בקרת כניסה

בנוסף המערכת סוגרת את הדלת מיידיית בלחיצה על מקש '*' בקודן, ופותחת את הדלת מיידיית בלחיצה על לחצן שבתוך החממה.

5.1.7 פונקציות לבקרת טמפרטורה

פונקציות אלו מטפלות במידע המתקבל מחיישני הטמפרטורה, ומתפעלות את מערכת האווירור בחממה.

AnalogToTemp 5.1.7.1

פונקציה זו ממירה את הערך שהתקבל מהחיישן דרך ה-A/D של הארדווייז וממירה אותו למתח אנלוגי, ולאחר מכן לטמפרטורה. לפי הנוסחה בסעיף 3.5.5

TempMeasurement 5.1.7.2

פונקציה זו קולטת את הערך מ-2 חיישני הטמפרטורה ושולחת כל ערך בנפרד לפונקציה *AnalogToTemp*, ולאחר מכן ממצעת בין 2 הערכים. ממוצע הטמפרטורות שהתקבל מסונן דרך פילטר תוכנתי אשר מסנן שינויי טמפרטורה קיצוניים. הפילטר התוכנתי מממצע 10 תוצאות טמפרטורה אחרונות, ממוצע זה משמש כ- low pass filter דיגיטלי פשוט.

הערך המסונן נשלח ל-GreenSee. בנוסף, הפונקציה קוראת לפונקציות הדלקת וכיבוי המאווררים בהתאם לטמפרטורה המסוננת לעומת ספי הטמפרטורות (הגבוהה והנמוכה) שהתקבלו מהמשתמש.

5.1.8 פונקציות לבקרת תאורה

פונקציות אלו מטפלות במידע המתקבל מחיישן ה-LDR, ומתפעלות את מערכת התאורה של החממה.

LightLEDsAfterShutter 5.1.8.1

פונקציה זו קוראת לפונקציה המדליקה את סרט הלדים, אם עוצמת האור המסוננת קטנה מעוצמת האור המוגדרת כנמוכה ע"י המשתמש. פונקציה זו נקראת כאשר יש צורך בתאורת יום, אך התריס המורם לא מכניס מספיק אור.

ShutterDownAfterLEDs 5.1.8.2

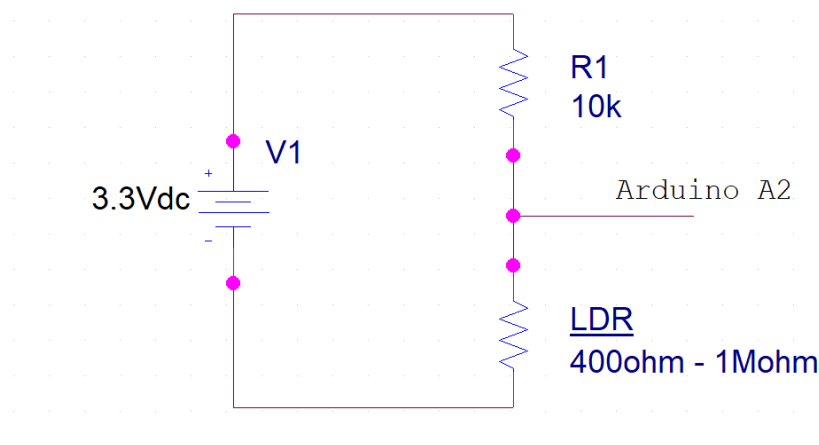
פונקציה זו קוראת לפונקציה המורידה את התריס. היא נקראת כאשר יש צורך בחושך בחממה.

LuminosityMeasurement 5.1.8.3

זו הפונקציה הראשית לבקרה של התאורה.

ראשית הפונקציה מבצעת קריאה של מתח מחלק המתח של ה-LDR

פונקציה זו קולטת את עוצמת האור הנוכחית מחיישן ה-LDR וממפה את הערכים הנקלטים בין 0 ל-1023.



תרשים 31: מעגל ה-LDR

מעגל ה-LDR הוא מחלק מתח אשר בהתאם להתנגדות ה-LDR מודד את עוצמת האור ב-LUX. לפי דף הנתונים של ה-LDR ערכי ההתנגדות הם בהתאם לטבלה הבאה:

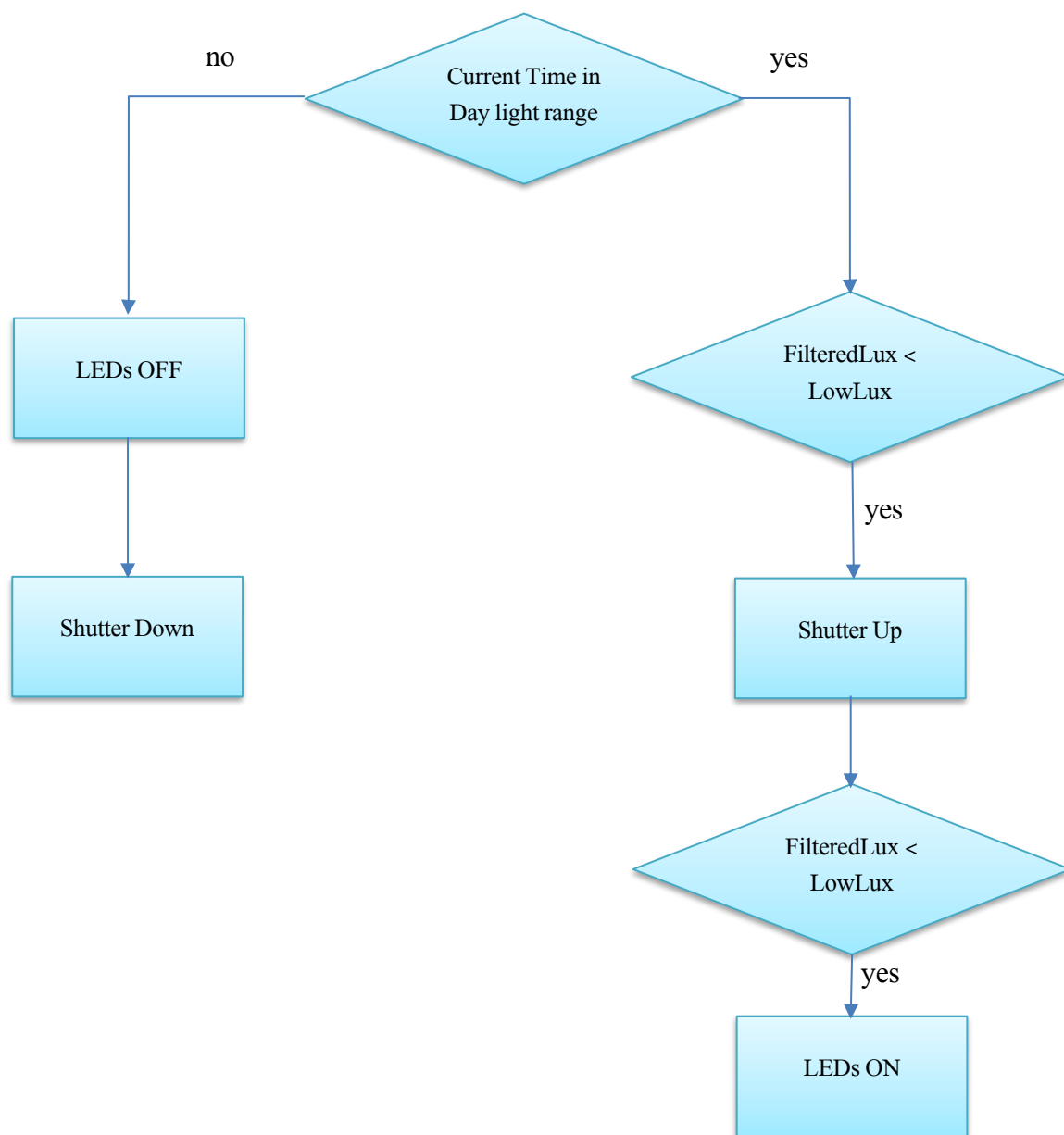
טבלה 12: טבלת המרה מערך A/D ל-LUX

עוצמת תאורה	התנגדות	מתח בפין A2 (מחלק מתח)	ערך ב-A/D
0 LUX	1MΩ	3.26V	1013
10LUX	9KΩ	1.56V	485
1000LUX	400Ω	0.12V	39

החלטנו למפות מערך A/D לערך LUX ע"י 2 המרות לינאריות: אחת עבור ערכי A/D 39 עד 485, והשני מ-485 עד 1013.

```
if(ldrsensor < 485)
  CurrentLuminosity = map(ldrsensor, 39, 485, 1000, 10);
else
  CurrentLuminosity = map(ldrsensor, 485, 1013, 10, 0);
```

לערך הנמדד מבצעים סינון באופן דומה לסינון שמבצעים לטמפרטורה. המיצוע בוצע ל-50 דגימות. בקרת התאורה שולטת על התריס ועל סרט הלדים בהתאם לעוצמת התאורה. בכל יום מוגדר בפרוטוקול הצמח טווח שעות לתאורת הצמח, שאר השעות הצמח צריך להיות בחשכה. הבקרה שיישמונו מתוארת בתרשים הזרימה הבא:



תרשים 32 : תרשים זרימה בקרת תאורה

```

#include <Keypad.h>

/* Defines */
#define TEMPERATURE_FILTER_LEN 10
#define LUX_FILTER_LEN 50
#define GREEN_LINE_MAX_RX_BUFFER 256

#define DOOR_EN_PWM 128
#define SHUTTER_UP_PWM 75
#define SHUTTER_DOWN_PWM 69

/* Arduino Pins */
const float aref_voltage = 3.3;
const int EntranceDoor_ENA = 2;
const int WaterValve_ENB = 3;
const int Shutter_ENA = 4;
const int DoorPushButton = 29;
const int EntranceDoor_in1 = 30;
const int EntranceDoor_in2 = 31;
const int Fan_in1 = 32;
const int Fan_in2 = 33;
const int Fan_in3 = 34;
const int Fan_in4 = 35;
const int WaterValve_in3 = 36;
const int WaterValve_in4 = 37;
const int Shutter_in1 = 38;
const int Shutter_in2 = 39;
const int Lights_in = 40;
const int TempSense1 = A0;
const int TempSense2 = A1;
const int LightSense = A2;

/* Local-time variables */
unsigned int Second = 0;
unsigned int Minute = 0;
unsigned int Hour = 10;
unsigned int Day = 4;

/* Software timers variables */
const unsigned int Timer4OneSecondPreload = 3036; // preload timer 65536-16MHz/256/1Hz
const unsigned int NumTimers = 15;
struct SoftTimer
{
    bool timerEnable;
    unsigned int durationSec;
    void (*callBackFunction)();
};

enum TimersNames
{
    CloseValve=0,
    CloseDoorDelay=1,
    WrongDoorKeyLock=2,
    DoorDriver=3,
    ShutterDriver=4,
    WaterValveDriver=5,
    CheckLightAfterShutterUp=6,
    CheckLightAfterLEDOff=7
};

struct SoftTimer Timers[15] =
{{false,0,NULL},{false,0,NULL},{false,0,NULL},{false,0,NULL},{false,0,NULL},{false,0,NUL
L},{false,0,NULL},{false,0,NULL},{false,0,NULL},{false,0,NULL},{false,0,NUL
L},{false,0,NULL}};

/* Configuration variables */
struct PlantDayProtocol
{
    bool dayEnable;

```

```

    unsigned int irrigationHour;
    unsigned int irrigationMinute;
    unsigned int duration;
    unsigned int lightStartHour;
    unsigned int lightStartMinute;
    unsigned int lightEndHour;
    unsigned int lightEndMinute;
};
float PlantDryThreshold = 1.0;

struct PlantDayProtocol MyPlantProtocol[7] =
{{true,8,0,600,6,0,20,0},{false,8,0,600,6,0,20,0},{true,8,0,600,6,0,20,0},{false,8,0,600,6,0,20,0},{true,8,0,600,6,0,20,0},{false,8,0,600,6,0,20,0},{true,8,0,600,6,0,20,0}};
int LowTemperature=0;
int HighTemperature=200;
int LowLux = 300;
int ShutterUpDelay = 6;
int ShutterDownDelay = 7;

/* GreenLine variables */
char GLMsgBuffer[GREEN_LINE_MAX_RX_BUFFER];

enum GreenLine_state
{
    SearchSOM, //search start of message
    CollectMsg
};

enum GreenLine_state GLState = SearchSOM;
int GLMsgBufferIndex = 0;

/* Sensors data & Variables */
float TensiometerVoltage;
float BattetyVoltage;
int CurrentTemperature;
int TempFilter[TEMPERATURE_FILTER_LEN] = {0};
int TempFilterIndex = 0;
int TempFilterNumSamples = 0;
int TempFilterSum = 0;
int FilteredTemperature;
int CurrentLuminosity; //in LUX units
int LuxFilter[LUX_FILTER_LEN] = {0};
int LuxFilterIndex = 0;
int LuxFilterNumSamples = 0;
int LuxFilterSum = 0;
int FilteredLux;

/* State variables */
bool FanIsOn = true;
bool LightIsOn = true;
bool ShutterIsUp = true;
bool MakeItDarkerOneShot = true;
bool MakeItBrighterOneShot = true;
bool TimeOfDayNotInitialized = true;

/* XBee variables */
const byte START_DELIMETER = 0x7E;
const byte AT_COMMAND = 0x08;
byte XbeeRxMsg[50];
unsigned int MsgLength;
byte RxByte;
int XbeeRxMsgIndex = 0;
unsigned int RemoteAD0;
unsigned int RemoteAD1;
unsigned int XbeeRSSI;
const byte PanID[] = {0x31, 0x88};
const byte DestinationLow[] = {0x00, 0x00, 0x56, 0x78};
const byte MyAdress[] = {0x12, 0x34};
const byte CoordinatorEn[] = {0x01};
const byte APIEn[] = {0x01};

```

```

const byte AESEn[] = {0x01};
const byte AESKey[] = {0x52, 0x6F, 0x66, 0x6F, 0x72, 0x4C, 0x61, 0x6E, 0x64, 0x33, 0x31, 0x38, 0x38,
0x37, 0x39, 0x32};

enum rx_state
{
    FindDelimiter,
    SetLengthHigh,
    SetLengthLow,
    GetMsg
};

struct MsgAPI {
    byte StartDelimiter;
    unsigned int Length;
    byte FrameType;
    byte FrameID;
    char ATCommand[2];
    byte ParameterValue[20];
};

rx state RxStateMachine = FindDelimiter;

/* Door and keypad variables */
char* password = "3188";

bool WrongKeyLock = false;
bool doorIsLocked;
bool DoorIsClosed = true;
bool IncorrectPassword = false;

const int WrongKeyDelaySeconds = 30;
const int DoorOpenDelaySeconds = 10;
const int WrongTrialsBeforeLock = 3;
int WrongTrialsCnt = WrongTrialsBeforeLock;

int PasswordIndex = 0;

const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

byte rowPins[ROWS] = {41, 42, 43, 44}; //Four left sockets of the keypad
byte colPins[COLS] = {22, 23, 24, 25}; //Four right sockets of the keypad

Keypad keypad = Keypad( makeKeymap(keys), rowPins, colPins, ROWS, COLS );

/***** Local Time Functions *****/

void SendLocalTime()
{
    char TimeString[10];

    Serial.print("~T=");
    sprintf(TimeString, "%02d:%02d", Hour, Minute);
    Serial.println(TimeString);
}

void HandleDayClock()
{
    Second++;
    if (Second == 60)
    {
        Second = 0;
        Minute++;
    }
}

```

```

    if (Minute == 60) {Minute = 0; Hour++;}
    if (Hour == 24) {Hour = 0; Day++;}
    if (Day == 7) Day = 0;
    SendLocalTime();
}
}

/** On/Off functions to the GreenHouse systems */

void WaterValveDriverDisable()
{
    digitalWrite(WaterValve_ENB, LOW);
}

void OpenWaterValve()
{
    Serial.println("~D=Open Water Valve");
    digitalWrite(WaterValve_ENB, HIGH);
    digitalWrite(WaterValve_in3, HIGH);
    digitalWrite(WaterValve_in4, LOW);
    SetSoftTimer(WaterValveDriver, 2, &WaterValveDriverDisable);
}

void CloseWaterValve()
{
    Serial.println("~D=Close Water Valve");
    digitalWrite(WaterValve_ENB, HIGH);
    digitalWrite(WaterValve_in3, LOW);
    digitalWrite(WaterValve_in4, HIGH);
    SetSoftTimer(WaterValveDriver, 2, &WaterValveDriverDisable);
}

void CloseWaterValveSetup()
{
    Serial.println("~D=Close Water Valve");
    digitalWrite(WaterValve_ENB, HIGH);
    digitalWrite(WaterValve_in3, LOW);
    digitalWrite(WaterValve_in4, HIGH);
    delay(1000);
    analogWrite(WaterValve_ENB, 0);
}

void DoorDriverDisable()
{
    analogWrite(EntranceDoor_ENA, 0);
}

void OpenDoor()
{
    PasswordIndex = 0;
    Serial.println("~D=Door opened");
    analogWrite(EntranceDoor_ENA, DOOR_EN_PWM);
    digitalWrite(EntranceDoor_in1, HIGH);
    digitalWrite(EntranceDoor_in2, LOW);
    SetSoftTimer(DoorDriver, 2, &DoorDriverDisable);
    DoorIsClosed = false;
}

void CloseDoor()
{
    PasswordIndex = 0;
    Serial.println("~D=Door closed");
    analogWrite(EntranceDoor_ENA, DOOR_EN_PWM);
    digitalWrite(EntranceDoor_in1, LOW);
    digitalWrite(EntranceDoor_in2, HIGH);
    SetSoftTimer(DoorDriver, 2, &DoorDriverDisable);
    DoorIsClosed = true;
}

void CloseDoorSetup()
{

```

```

    PasswordIndex = 0;
    Serial.println("~D=Door closed");
    analogWrite(EntranceDoor_ENA, DOOR_EN_PWM);
    digitalWrite(EntranceDoor_in1, LOW);
    digitalWrite(EntranceDoor_in2, HIGH);
    delay(1000);
    analogWrite(EntranceDoor_ENA, 0);
    DoorIsClosed = true;
}

void FansOn()
{
    if (FanIsOn)
        return;
    FanIsOn = true;
    Serial.println("~D=fans on");
    digitalWrite(Fan_in1, LOW);
    digitalWrite(Fan_in2, LOW);
    digitalWrite(Fan_in3, LOW);
    digitalWrite(Fan_in4, LOW);
}

void FansOff()
{
    if (!FanIsOn)
        return;
    FanIsOn = false;
    Serial.println("~D=fans off");
    digitalWrite(Fan_in1, HIGH);
    digitalWrite(Fan_in2, HIGH);
    digitalWrite(Fan_in3, HIGH);
    digitalWrite(Fan_in4, HIGH);
}

void LightsOn()
{
    if (LightIsOn)
        return;
    LightIsOn = true;
    Serial.println("~D=Lights on");
    digitalWrite(Lights_in, LOW);
}

void LightsOff()
{
    if (!LightIsOn)
        return;
    LightIsOn = false;
    Serial.println("~D=Lights off");
    digitalWrite(Lights_in, HIGH);
}

void ShutterDriverDisable()
{
    analogWrite(Shutter_ENA, 0);
}

void ShutterUp()
{
    if (ShutterIsUp)
        return;
    ShutterIsUp = true;
    Serial.println("~D=shutter up");
    analogWrite(Shutter_ENA, SHUTTER_UP_PWM);
    digitalWrite(Shutter_in1, HIGH);
    digitalWrite(Shutter_in2, LOW);
    SetSoftTimer(ShutterDriver, ShutterUpDelay+1, &ShutterDriverDisable);
}

void ShutterDown()
{

```

```

    if (!ShutterIsUp)
        return;
    ShutterIsUp = false;
    Serial.println("~D=shutter down");
    analogWrite(Shutter_ENA, SHUTTER_DOWN_PWM);
    digitalWrite(Shutter_in1, LOW);
    digitalWrite(Shutter_in2, HIGH);
    SetSoftTimer(ShutterDriver, ShutterDownDelay+1, &ShutterDriverDisable);
}

/** Software Timers functions */
void SetSoftTimer(enum TimersNames timerName, unsigned int duration, void (*ptr)())
{
    Timers[timerName].durationSec = duration;
    Timers[timerName].timerEnable = true;
    Timers[timerName].callBackFunction = ptr;
}

void DisableSoftTimer(enum TimersNames timerName)
{
    Timers[timerName].timerEnable = false;
}

void CheckSoftTimers()
{
    for (int i=0; i<NumTimers; i++)
    {
        if (Timers[i].timerEnable)
        {
            Timers[i].durationSec--;
            if (Timers[i].durationSec == 0)
            {
                Timers[i].timerEnable = false;
                (*Timers[i].callBackFunction) ();
            }
        }
    }
}

/** GreenLine functions */
void GreenSeeHandler()
{
    // handle GreenSee communication
    if (Serial.available())
    { // If data comes in from GreenSee application, coollect it into a buffer
        char gLRxChar = Serial.read();
        GLStateMachine(gLRxChar);
    }
}

void GLStateMachine(char rxChar)
{
    switch(GLState)
    {
        case SearchSOM:
            if(rxChar == '~')
                GLState = CollectMsg;
            GLMsgBufferIndex = 0;
            break;
        case CollectMsg:
            GLMsgBuffer[GLMsgBufferIndex] = rxChar;
            GLMsgBufferIndex++;
            // check buffer overflow
            if (GLMsgBufferIndex == GREEN_LINE_MAX_RX_BUFFER)
            {
                Serial.println("~D=Got from GreenSee too big message");
                GLState = SearchSOM;
            }
            if(rxChar == 10) //new line character, indicates end of message
            {

```



```

        GLState = SearchSOM;
        GLParseMsg();
    }
    break;
}
}

void GLParseMsg()
{
    String s = "";
    int ind1, ind2, ind3, ind4, ind5, ind6, ind7, ind8;
    int day;
    String dayS = "", dayEnS = "", hourS = "", minuteS = "", durationS = "", localDayS = "", localHourS =
    "", localMinuteS = "", localSecondS = "", hourS2 = "", minuteS2 = "", hourS3 = "", minuteS3 = "" ;

    for(int i = 2; i < GLMsgBufferIndex; i++)
        s += GLMsgBuffer[i];

    switch(GLMsgBuffer[0])
    {
        case 'I':
            ind1 = s.indexOf(',');
            dayS = s.substring(0, ind1);
            ind2 = s.indexOf(',', ind1+1);
            dayEnS = s.substring(ind1+1, ind2);
            ind3 = s.indexOf(',', ind2+1);
            hourS = s.substring(ind2+1, ind3);
            ind4 = s.indexOf(',', ind3+1);
            minuteS = s.substring(ind3+1, ind4);
            ind5 = s.indexOf(',', ind4+1);
            durationS = s.substring(ind4+1, ind5);
            ind6 = s.indexOf(',', ind5+1);
            hourS2 = s.substring(ind5+1, ind6);
            ind7 = s.indexOf(',', ind6+1);
            minuteS2 = s.substring(ind6+1, ind7);
            ind8 = s.indexOf(',', ind7+1);
            hourS3 = s.substring(ind7+1, ind8);
            minuteS3 = s.substring(ind8+1);
            day = dayS.toInt();
            MyPlantProtocol[day].dayEnable = dayEnS.toInt();
            MyPlantProtocol[day].irrigationHour = hourS.toInt();
            MyPlantProtocol[day].irrigationMinute = minuteS.toInt();
            MyPlantProtocol[day].duration = durationS.toInt();
            MyPlantProtocol[day].lightStartHour = hourS2.toInt();
            MyPlantProtocol[day].lightStartMinute = minuteS2.toInt();
            MyPlantProtocol[day].lightEndHour = hourS3.toInt();
            MyPlantProtocol[day].lightEndMinute = minuteS3.toInt();
            break;
        case 'L':
            LowTemperature = s.toInt();
            break;
        case 'H':
            HighTemperature = s.toInt();
            break;
        case 'U':
            PlantDryThreshold = s.toFloat();
            break;
        case 'V':
            LowLux = s.toInt();
            break;
        case 'Y':
            ShutterUpDelay = s.toInt();
            break;
        case 'Z':
            ShutterDownDelay = s.toInt();
            break;
        case 'T':
            ind1 = s.indexOf(',');
            localDayS = s.substring(0, ind1);
            ind2 = s.indexOf(',', ind1+1);
            localHourS = s.substring(ind1+1, ind2);

```

```

ind3 = s.indexOf(',', ind2+1 );
localMinuteS = s.substring(ind2+1, ind3);
localSecondS = s.substring(ind3+1);
Day = localDayS.toInt();
Hour = localHourS.toInt();
Minute = localMinuteS.toInt();
Second = localSecondS.toInt();
SendLocalTime();
TimeOfDayNotInitialized = false;
break;
case 'X':
    // GreenSee send tests
    int testNum = s.toInt();
    switch(testNum)
    {
        case 1:
            FansOn();
            break;
        case 2:
            FansOff();
            break;
        case 3:
            LightsOn();
            break;
        case 4:
            LightsOff();
            break;
        case 5:
            ShutterUp();
            break;
        case 6:
            ShutterDown();
            break;
        case 7:
            OpenWaterValve();
            break;
        case 8:
            CloseWaterValve();
            break;
        default:
            Serial.println("~D=Unkonwn Test number " + s);
            break;
    }
    break;
default:
    Serial.println("~D=Unkonwn GreenSee Msg " + s);
    break;
}
}

/**/ XBee functions /**/

unsigned int EndianSwap(int x)
{
    byte MSByte;
    byte LSByte;
    LSByte = x >> 8;
    MSByte = x;
    return ((MSByte << 8) + LSByte);
}

void SendAPIMsg(byte *msg, int len)
{
    byte Checksum = 0;
    Serial.print("~D=AT command send: ");
    for (int i=0;i<len;i++)
    {
        Serial1.write(msg[i]);
        if(i>2)
            Checksum += msg[i];
        Serial.print(msg[i], HEX);
    }
}

```

```

    Serial.print(' ');
}
Checksum = 0xff - Checksum;
Serial1.write(Checksum);
Serial.print(Checksum, HEX);
Serial.println(' ');
delay(200);

Serial.print("~D=AT command response: ");

while(Serial1.available())
{
    Serial.print(Serial1.read(), HEX);
    Serial.print(' ');
}
Serial.println(' ');
}

void PrepareATCommand(struct MsgAPI* msg, char *ATString, byte *params, int paramLength)
{
    msg->StartDelimiter = START_DELIMITER;
    msg->Length = EndianSwap(paramLength + 4);
    msg->FrameType = AT_COMMAND;
    msg->FrameID = 1;
    msg->ATCommand[0] = ATString[0];
    msg->ATCommand[1] = ATString[1];
    memcpy(msg->ParameterValue, params, paramLength);
}

void ConfigureXBee()
{
    struct MsgAPI TxMsgATCommand;

    PrepareATCommand(&TxMsgATCommand, "FR", NULL, 0);
    SendAPIMsg((byte*) &TxMsgATCommand, 7);

    PrepareATCommand(&TxMsgATCommand, "ID", PanID, sizeof(PanID));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(PanID) + 7);

    PrepareATCommand(&TxMsgATCommand, "DL", DestinationLow, sizeof(DestinationLow));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(DestinationLow) + 7);

    PrepareATCommand(&TxMsgATCommand, "MY", MyAdress, sizeof(MyAdress));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(MyAdress) + 7);

    PrepareATCommand(&TxMsgATCommand, "CE", CoordinatorEn, sizeof(CoordinatorEn));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(CoordinatorEn) + 7);

    PrepareATCommand(&TxMsgATCommand, "AP", APIEn, sizeof(APIEn));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(APIEn) + 7);

    PrepareATCommand(&TxMsgATCommand, "EE", AESEn, sizeof(AESEn));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(AESEn) + 7);

    PrepareATCommand(&TxMsgATCommand, "KY", AESKey, sizeof(AESKey));
    SendAPIMsg((byte*) &TxMsgATCommand, sizeof(AESKey) + 7);

    PrepareATCommand(&TxMsgATCommand, "WR", NULL, 0);
    SendAPIMsg((byte*) &TxMsgATCommand, 7);
}

void XBeeHandler()
{
    while (Serial1.available()) //XBee/UART1/pins 18 and 19
    {
        RxByte = Serial1.read();
        switch(RxStateMachine)
        {
            case FindDelimiter:
                if(RxByte == START_DELIMITER)
                {

```

```

        RxStateMachine = SetLengthHigh;
        XbeeRxMsgIndex = 0;
    }
    break;

case SetLengthHigh:
    MsgLength = RxByte << 8;
    RxStateMachine = SetLengthLow;
    break;

case SetLengthLow:
    MsgLength += RxByte;
    RxStateMachine = GetMsg;
    break;

case GetMsg:
    XbeeRxMsg[XbeeRxMsgIndex++] = RxByte;
    MsgLength--;
    if(MsgLength == 0)
    {
        RxStateMachine = FindDelimiter;

        XbeeRSSI = XbeeRxMsg[3];
        Serial.print("~R=");
        Serial.println(XbeeRSSI);

        RemoteAD0 = (XbeeRxMsg[8] << 8) + XbeeRxMsg[9];
        Serial.print("~D=AD0=");
        Serial.print(RemoteAD0);
        TensiometerVoltage = (map(RemoteAD0, 0, 1023, 0, 3300))/1000.0;
        Serial.print(" Tensiometer voltage=");
        Serial.println(TensiometerVoltage, 3);
        Serial.print("~M=");
        Serial.println(TensiometerVoltage, 3);

        RemoteAD1 = (XbeeRxMsg[10] << 8) + XbeeRxMsg[11];
        Serial.print("~D=AD1=");
        Serial.print(RemoteAD1);
        BattetyVoltage = ((map(RemoteAD1, 0, 1023, 0, 3300))/1000.0)*3;
        Serial.print(" Battery voltage=");
        Serial.println(BattetyVoltage, 3);
        Serial.print("~B=");
        Serial.println(BattetyVoltage, 3);
        if(BattetyVoltage < 7.5)
            Serial.println("~D=Low Battery!");
    }
    break;
}
}
}

/** Irrigation plan function */
void CheckIrrigationPlan()
{
    if (TimeOfDayNotInitialized)
        return;
    if(MyPlantProtocol[Day].dayEnable && (MyPlantProtocol[Day].irrigationHour == Hour) &&
(MyPlantProtocol[Day].irrigationMinute == Minute) && (Second == 0) && (TensiometerVoltage >
PlantDryThreshold))
    {
        OpenWaterValve();
        SetSoftTimer(CloseValve, MyPlantProtocol[Day].duration+1, &CloseWaterValve);
    }
}

/** Door System functions */
void KeyLockRelease()
{
    WrongKeyLock = false;
    WrongTrialsCnt = WrongTrialsBeforeLock;
}

```

```

    Serial.println("~D=Key lock released, reenter password");
}

void GreenHouseDoorHandler()
{
    char key = keypad.getKey();

    // Check key pressed and open door if code is ok
    if(key && !WrongKeyLock)
    {
        if (key == '*')
        {
            DisableSoftTimer(CloseDoorDelay);
            CloseDoor();
            PasswordIndex = 0;
            IncorrectPassword = false;
        }
        else
        {
            if(key == password[PasswordIndex])
            {
                PasswordIndex++;
            }
            else
            {
                IncorrectPassword = true;
                PasswordIndex++;
            }
        }
    }

    // check if password ok after 4 keys pressed
    if(PasswordIndex == 4)
    {
        if(IncorrectPassword)
        {
            WrongTrialsCnt--;
            PasswordIndex = 0;
            IncorrectPassword = false;
            Serial.print("~D=Wrong password trials left: ");
            Serial.println(WrongTrialsCnt);
            if(WrongTrialsCnt == 0)
            {
                WrongKeyLock = true;
                Serial.println("~D=Wrong Key Lock, wait 30 seconds");
                SetSoftTimer(WrongDoorKeyLock, WrongKeyDelaySeconds+1, &KeyLockRelease);
            }
        }
        else
        {
            OpenDoor();
            SetSoftTimer(CloseDoorDelay, DoorOpenDelaySeconds+1, &CloseDoor);
        }
    }
}

// internal push button pressed - open door
if(!digitalRead(DoorPushButton) && DoorIsClosed)
{
    OpenDoor();
    SetSoftTimer(CloseDoorDelay, DoorOpenDelaySeconds+1, &CloseDoor);
}

}

/** Temperature control functions */
float AnalogToTemp(int tempReading)
{
    float voltage = (tempReading * aref_voltage)/1023.0;
    float temperature = (voltage - 0.5) * 100.0 ;
    return(temperature);
}

```

```

void TempMeasurement()
{
    int tempReading1 = analogRead(TempSense1);
    int tempReading2 = analogRead(TempSense2);
    float temp1 = AnalogToTemp(tempReading1);
    float temp2 = AnalogToTemp(tempReading2);
    CurrentTemperature = (int) ((temp1 + temp2)/2 + 0.5);

    // Filter the temperature by running avvarage
    TempFilterSum -= TempFilter[TempFilterIndex];
    TempFilterSum += CurrentTemperature;
    TempFilter[TempFilterIndex] = CurrentTemperature;
    TempFilterIndex++;
    if(TempFilterIndex == TEMPERATURE_FILTER_LEN)
    {
        TempFilterIndex = 0;
    }
    if(TempFilterNumSamples < TEMPERATURE_FILTER_LEN)
        TempFilterNumSamples++;

    // finally the filtered temprature is the average of all samples
    FilteredTemperature = (TempFilterSum + TempFilterNumSamples/2.0) / TempFilterNumSamples;

    Serial.print("~C=");
    Serial.println(FilteredTemperature);

    if (FilteredTemperature >= HighTemperature)
    {
        FansOn();
    }

    if (FilteredTemperature <= LowTemperature)
    {
        FansOff();
    }
}

/** Light control functions */
void LightLEDsAfterShutter()
{
    if (FilteredLux < LowLux)
    {
        LightsOn();
    }
}

void ShutterDownAfterLEDs()
{
    ShutterDown();
}

void LuminosityMeasurement()
{
    int currentDayMinutes = Hour * 60 + Minute;
    int lightStartDayMinutes = MyPlantProtocol[Day].lightStartHour * 60 +
    MyPlantProtocol[Day].lightStartMinute;
    int lightEndDayMinutes = MyPlantProtocol[Day].lightEndHour * 60 +
    MyPlantProtocol[Day].lightEndMinute;

    int ldrsensor=analogRead(LightSense);
    if(ldrsensor < 485)
        CurrentLuminosity = map(ldrsensor, 39, 485, 1000, 10);
    else
        CurrentLuminosity = map(ldrsensor, 485, 1013, 10, 0);

    // Filter the Lux by running avvarage

    LuxFilterSum -= LuxFilter[LuxFilterIndex];
    LuxFilterSum += CurrentLuminosity;

```

```

LuxFilter[LuxFilterIndex] = CurrentLuminosity;
LuxFilterIndex++;
if(LuxFilterIndex == LUX_FILTER_LEN)
{
    LuxFilterIndex = 0;
}
if(LuxFilterNumSamples < LUX_FILTER_LEN)
    LuxFilterNumSamples++;

// finally the filtered temprature is the average of all samples
FilteredLux = (LuxFilterSum + LuxFilterNumSamples/2.0) / LuxFilterNumSamples;

Serial.print("~L=");
Serial.println(FilteredLux);

// Light automation
if (TimeOfDayNotInitialized)
    return;

if(currentDayMinutes > lightStartDayMinutes && currentDayMinutes < lightEndDayMinutes)
{
    /* Plant need light */
    MakeItDarkerOneShot = true;
    if (FilteredLux < LowLux && MakeItBrighterOneShot)
    {
        MakeItBrighterOneShot = false;
        ShutterUp();
        SetSoftTimer(CheckLightAfterShutterUp, ShutterUpDelay + LUX_FILTER_LEN, &LightLEDsAfterShutter);
    }
}
else
{
    MakeItBrighterOneShot = true;
    /* Plant need darkness */
    if (MakeItDarkerOneShot)
    {
        MakeItDarkerOneShot = false;
        LightsOff();
        SetSoftTimer(CheckLightAfterLEDOff, LUX_FILTER_LEN, &ShutterDownAfterLEDs);
    }
}
}

/** Setup function */
void setup()
{
    pinMode(Fan_in1, OUTPUT);
    pinMode(Fan_in2, OUTPUT);
    pinMode(Fan_in3, OUTPUT);
    pinMode(Fan_in4, OUTPUT);
    pinMode(Lights_in, OUTPUT);
    FansOff();
    LightsOff();
    Serial.begin(9600);
    Serial1.begin(9600); //XBee/UART1/pins 18 and 19
    Serial.println("~D=Hello GreenSee");
    pinMode(WaterValve_ENB, OUTPUT);
    pinMode(WaterValve_in3, OUTPUT);
    pinMode(WaterValve_in4, OUTPUT);
    pinMode(EntranceDoor_ENA, OUTPUT);
    pinMode(EntranceDoor_in1, OUTPUT);
    pinMode(EntranceDoor_in2, OUTPUT);

    pinMode(Shutter_ENA, OUTPUT);
    pinMode(Shutter_in1, OUTPUT);
    pinMode(Shutter_in2, OUTPUT);
    analogReference(EXTERNAL);
    pinMode(DoorPushButton, INPUT_PULLUP);

    CloseWaterValveSetup();
    CloseDoorSetup();
}

```

```
ConfigureXBee();

Serial.println("~D=GreenDo V1.5 3/5/2019 Ready");

// initialize timer4
noInterrupts(); // disable all interrupts
TCCR4A = 0;
TCCR4B = 0;

TCNT4 = Timer4OneSecondPreload;
TCCR4B |= (1 << CS12) ; // 256 prescaler
TIMSK4 |= (1 << TOIE4); // enable timer overflow interrupt
interrupts(); // enable all interrupts
}

/** Interrupt Service Routine */
ISR(TIMER4_OVF_vect)
{
  TCNT4 = Timer4OneSecondPreload;
  HandleDayClock();
  CheckIrrigationPlan();
  TempMeasurement();
  LuminosityMeasurement();
  CheckSoftTimers();
  XBeeHandler();
}

/** Loop function */
void loop()
{
  GreenSeeHandler();
  GreenHouseDoorHandler();
}
```

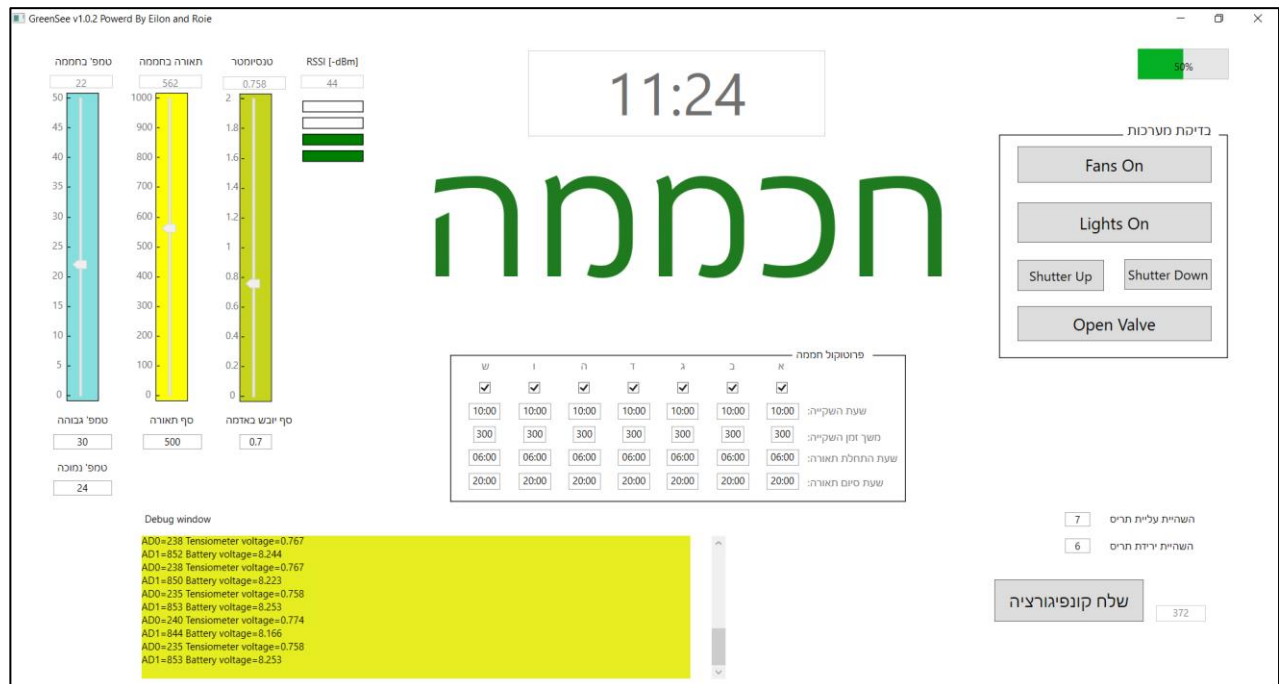

5.3 אפליקציית GreenSee

אפליקציית GreenSee היא אפליקציה שמופעלת במחשב. האפליקציה משרתת 3 מטרות:

1. שליחת קונפיגורציה ל-GreenDo כגון פרוטוקול הצמח ופרמטרים נוספים

2. תצוגה של מצב החיישנים השונים במערכת

3. ביצוע Test למערכות השונות וחלון להודעות debug של ה-GreenDo



תרשים 33: צילום מסך של תוכנת GreenSee

האפליקציה נכתבה בשפת C# בסביבת WPF של Visual Studio 2017. כיוון שהתוכנה אינה עיקר הפרויקט, אלא חלק משני, לא נתאר תיאור מפורט של התוכנה, אלא רק תיאור של הפונקציות העיקריות. באפליקציה מפעילים טיימר תוכנתי כל 100mSec וכל הפונקציות מתבצעות מהפונקציה של טיימר זה.

5.3.1 MainWindow

פונקציה זו מאתחלת את ערוץ התקשורת בין המחשב לארדואינו ואת הטיימר הראשי.

5.3.2 ParseMsg

פונקציה זו מנתחת את מחרוזת ההודעה שנקלטה מהארדואינו ומציגה את הנתונים בהתאם לסוג ההודעה. סוגי ההודעות מפורטות בטבלה 10.

5.3.3 GreenLineStateMachine

פונקציה זו מממשת מכונת מצבים לקליטת הודעה מה-GreenDo בדומה למכונת המצבים המתוארת ב-5.1.4.2. GLStateMachine

MainTimerTick 5.3.4

פונקציית הטיימר שבודקת אם נקלט תו חדש בערוץ הטורי וקוראת לפונקציית GreenLineStateMachine.

ConfigurationButton_Click 5.3.5

פונקציה זו נקראת בלחיצה על כפתור "שלח קונפיגורציה" ושולחת הודעות של כל פרמטרי הקונפיגורציה בהתאם ל - טבלה 11.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

using System.IO.Ports;
using System.Windows.Threading;
using System.IO;

namespace GreenSee
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        // Put here global costatnats & Variables
        int counter = 0;
        bool TestsFansIsOn = false;
        bool TestsLightsIsOn = false;
        bool TestsValveIsOpen = false;
        int NoSignalCnt = 0;

        enum GreenLine_state
        {
            SearchSOM, //search start of message
            CollectMsg
        };

        GreenLine_state state = GreenLine_state.SearchSOM;
        private char[] MsgBuffer = new char[256];
        private int MsgBufferIndex = 0;

        // serial port to communicate with Arduino
        private SerialPort GreenLine = new SerialPort();

        // GreenSee main 100msec timer
        private DispatcherTimer MainTimer = new DispatcherTimer();

        public MainWindow()
        {
            InitializeComponent();

            // Initlize timer
            MainTimer.Interval = TimeSpan.FromMilliseconds(100);
            MainTimer.Tick += MainTimerTick;

            // from here start button code
            // start timer
            MainTimer.Start();

            //configure arduino COM port
            GreenLine.PortName = "COM7";
            GreenLine.BaudRate = 9600;
            GreenLine.Handshake = System.IO.Ports.Handshake.None;
            GreenLine.Parity = Parity.None;
            GreenLine.DataBits = 8;
        }
    }
}

```

```

GreenLine.StopBits = StopBits.One;
GreenLine.ReadBufferSize = 4096;
GreenLine.ReadTimeout = -1;
GreenLine.ReceivedBytesThreshold = 1;

GreenLine.Open();
GreenLine.DiscardInBuffer();

if (GreenLine.IsOpen == false)
    MessageBox.Show("לא ניתן לפתוח ערוץ סיריאל", "GreenSee");
}

void ParseMsg()
{
    string s = "";
    for(int i = 2; i < MsgBufferIndex; i++)
        s += MsgBuffer[i];
    switch(MsgBuffer[0])
    {
        case 'D':
            DebugBlock.Text += s;
            DebugScroll.ScrollToBottom();
            break;
        case 'C':
            CurrentTemprature.Text = s;
            TempratureSlider.Value = int.Parse(s);
            if(TempratureSlider.Value > int.Parse(TempratureHigh.Text))
                TempratureSlider.Background = Brushes.Red;
            else if(TempratureSlider.Value < int.Parse(TempratureLow.Text))
                TempratureSlider.Background = new SolidColorBrush(Color.FromArgb(0x7F, 0x0B,
0xBF, 0xBC));
            else
                TempratureSlider.Background = Brushes.LightGreen;
            break;
        case 'T':
            ArduinoTime.Text = s;
            break;
        case 'B':
            float val = float.Parse(s);
            if(val < 7.5)
                val = 7.5f;
            int batteryLevel = (int)((val - 7.5) / 1.5 * 100);
            BatteryLevel.Value = batteryLevel;
            BatteryLevelPercentage.Text = batteryLevel.ToString() + "%";
            break;
        case 'M':
            CurrentVoltage.Text = s;
            TensiometerSlider.Value = float.Parse(s);
            if (TensiometerSlider.Value > 0.7)
                TensiometerSlider.Background = new SolidColorBrush(Color.FromArgb(0xFF, 0xC6,
0xD4, 0x1C));
            else
                TensiometerSlider.Background = new SolidColorBrush(Color.FromArgb(0xFF, 0x26,
0xAE, 0x26));
            break;
        case 'L':
            CurrentLux.Text = s;
            LuxSlider.Value = int.Parse(s);
            if (LuxSlider.Value > int.Parse(LuxLow.Text))
                LuxSlider.Background = Brushes.Yellow;
            else
                LuxSlider.Background = new SolidColorBrush(Color.FromArgb(0xFF, 0x80, 0x80,
0x80));
            break;
        case 'R':
            RSSI.Text = s;
            NoSignalCnt = 0;
            int RSSIValue = int.Parse(s);

```

```

        if (RSSIValue > 50)
        {
            Rect4.Fill = Brushes.Green;
            Rect3.Fill = Brushes.White;
            Rect2.Fill = Brushes.White;
            Rect1.Fill = Brushes.White;
        }
        else if (RSSIValue > 40 )
        {
            Rect4.Fill = Brushes.Green;
            Rect3.Fill = Brushes.Green;
            Rect2.Fill = Brushes.White;
            Rect1.Fill = Brushes.White;
        }
        else if (RSSIValue > 30)
        {
            Rect4.Fill = Brushes.Green;
            Rect3.Fill = Brushes.Green;
            Rect2.Fill = Brushes.Green;
            Rect1.Fill = Brushes.White;
        }
        else
        {
            Rect4.Fill = Brushes.Green;
            Rect3.Fill = Brushes.Green;
            Rect2.Fill = Brushes.Green;
            Rect1.Fill = Brushes.Green;
        }

        break;
    default:
        MessageBox.Show("הודעה לא מוכרת", "GreenSee");
        break;
    }
}

void GreenLineStateMachine(char rxChar)
{
    switch(state)
    {
        case GreenLine_state.SearchSOM:
            if(rxChar == '~')
                state = GreenLine_state.CollectMsg;
            MsgBufferIndex = 0;
            break;
        case GreenLine_state.CollectMsg:
            MsgBuffer[MsgBufferIndex] = rxChar;
            MsgBufferIndex++;
            if(rxChar == 10) //new line character, indicates end of message
            {
                state = GreenLine_state.SearchSOM;
                ParseMsg();
            }
            break;
    }
}

void MainTimerTick(object sender, EventArgs e)
{
    int rxChar;
    NoSignalCnt++;
    if (NoSignalCnt >= 100)
    {
        NoSignalCnt = 100;
        RSSI.Text = "אִי קְלִיטָה";
        Rect4.Fill = Brushes.White;
        Rect3.Fill = Brushes.White;
        Rect2.Fill = Brushes.White;
        Rect1.Fill = Brushes.White;
    }
    counter++;
}

```

```

TimerText.Text = counter.ToString();
while(GreenLine.BytesToRead > 0)
{
    rxChar = GreenLine.ReadByte();
    GreenLineStateMachine((char) rxChar);
}

private void ConfigurationButton_Click(object sender, RoutedEventArgs e)
{
    string s;
    s = String.Format("~I=0,{0},{1},{2},{3},{4}", (bool) (SundayCheckbox.IsChecked) ? 1 : 0,
        SundayIrigationTime.Text.Replace(':', ' '), SundayIrigationDuration.Text,
        SundayLightStartTime.Text.Replace(':', ' '), SundayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~I=1,{0},{1},{2},{3},{4}", (bool) (MondayCheckbox.IsChecked) ? 1 : 0,
        MondayIrigationTime.Text.Replace(':', ' '), MondayIrigationDuration.Text,
        MondayLightStartTime.Text.Replace(':', ' '), MondayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~I=2,{0},{1},{2},{3},{4}", (bool) (TuesdayCheckbox.IsChecked) ? 1 : 0,
        TuesdayIrigationTime.Text.Replace(':', ' '), TuesdayIrigationDuration.Text,
        TuesdayLightStartTime.Text.Replace(':', ' '), TuesdayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~I=3,{0},{1},{2},{3},{4}", (bool) (WednesdayCheckbox.IsChecked) ? 1 : 0,
        WednesdayIrigationTime.Text.Replace(':', ' '), WednesdayIrigationDuration.Text,
        WednesdayLightStartTime.Text.Replace(':', ' '), WednesdayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~I=4,{0},{1},{2},{3},{4}", (bool) (ThursdayCheckbox.IsChecked) ? 1 : 0,
        ThursdayIrigationTime.Text.Replace(':', ' '), ThursdayIrigationDuration.Text,
        ThursdayLightStartTime.Text.Replace(':', ' '), ThursdayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~I=5,{0},{1},{2},{3},{4}", (bool) (FridayCheckbox.IsChecked) ? 1 : 0,
        FridayIrigationTime.Text.Replace(':', ' '), FridayIrigationDuration.Text,
        FridayLightStartTime.Text.Replace(':', ' '), FridayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~I=6,{0},{1},{2},{3},{4}", (bool) (SaturdayCheckbox.IsChecked) ? 1 : 0,
        SaturdayIrigationTime.Text.Replace(':', ' '), SaturdayIrigationDuration.Text,
        SaturdayLightStartTime.Text.Replace(':', ' '), SaturdayLightEndTime.Text.Replace(':', ' '));
    GreenLine.WriteLine(s);
    s = String.Format("~H={0}", TemperatureHigh.Text);
    GreenLine.WriteLine(s);
    s = String.Format("~L={0}", TemperatureLow.Text);
    GreenLine.WriteLine(s);
    s = String.Format("~V={0}", LuxLow.Text);
    GreenLine.WriteLine(s);
    s = String.Format("~U={0}", DryLevel.Text);
    GreenLine.WriteLine(s);
    DateTime localDate = DateTime.Now;
    int dayOfWeek = (int) localDate.DayOfWeek;
    s = String.Format("~T={0},{1},{2},{3}", dayOfWeek, localDate.Hour, localDate.Minute,
        localDate.Second);
    GreenLine.WriteLine(s);
    s = String.Format("~Y={0}", ShutterUpDelay.Text);
    GreenLine.WriteLine(s);
    s = String.Format("~Z={0}", ShutterDownDelay.Text);
    GreenLine.WriteLine(s);
}

private void FansTest_Click(object sender, RoutedEventArgs e)
{
    if (TestsFansIsOn)
    {
        FansTest.Background = new SolidColorBrush(Color.FromArgb(0xFF, 221, 221, 221));
        TestsFansIsOn = false;
        FansTest.Content = "Fans On";
        GreenLine.WriteLine("~X=2");
    }
    else
    {
        FansTest.Background = new SolidColorBrush(Color.FromArgb(0xFF, 212, 42, 42));
        TestsFansIsOn = true;
    }
}

```

```

        FansTest.Content = "Fans Off";
        GreenLine.WriteLine("~X=1");
    }
}

private void LightsTest_Click(object sender, RoutedEventArgs e)
{
    if (TestsLightsIsOn)
    {
        LightsTest.Background = new SolidColorBrush(Color.FromArgb(0xFF, 221, 221, 221));
        TestsLightsIsOn = false;
        LightsTest.Content = "Lights On";
        GreenLine.WriteLine("~X=4");
    }
    else
    {
        LightsTest.Background = new SolidColorBrush(Color.FromArgb(0xFF, 212, 42, 42));
        TestsLightsIsOn = true;
        LightsTest.Content = "Lights Off";
        GreenLine.WriteLine("~X=3");
    }
}

private void ShutterTestDown_Click(object sender, RoutedEventArgs e)
{
    GreenLine.WriteLine("~X=6");
}

private void ShutterTestUp_Click(object sender, RoutedEventArgs e)
{
    GreenLine.WriteLine("~X=5");
}

private void TestWaterValve_Click(object sender, RoutedEventArgs e)
{
    if (TestsValveIsOpen)
    {
        TestWaterValve.Background = new SolidColorBrush(Color.FromArgb(0xFF, 221, 221, 221));
        TestsValveIsOpen = false;
        TestWaterValve.Content = "Open Valve";
        GreenLine.WriteLine("~X=8");
    }
    else
    {
        TestWaterValve.Background = new SolidColorBrush(Color.FromArgb(0xFF, 212, 42, 42));
        TestsValveIsOpen = true;
        TestWaterValve.Content = "Close Valve";
        GreenLine.WriteLine("~X=7");
    }
}
}
}
}

```

6 איתור תקלות וקשיים בפרויקט

6.1 איתור תקלות

6.1.1 Timer Interrupt

לאחר שאתחלנו את הרגיסטרים הפסיקה של הטיימר עבדה רק פעם אחת ולא בצורה מחזורית. לאחר בדיקה באינטרנט, על פעולת הטיימרים הבנו שהטיימר מגיע ל-0xFFFF ואז מתרחשת פסיקה ועל-מנת שהפסיקה תתרחש שוב במחזוריות, צריך לטעון את רגיסטר מונה הטיימר לערך הראשוני

TCNT3 = Timer3OneSecondPreload

6.1.2 תוכנה נתקעה

לאחר דקה שהתוכנה עבדה היא נתקעה וכל הפונקציות שנקראות מתוך ה-loop הפסיקו לפעול, ורק פונקציות שנקראות מהפסיקה המשיכו לעבוד. כדי לאתר היכן התוכנה נתקעת הוספנו הודעות print לחלון ה-debug וראינו שהתוכנה נתקעת אחרי פונקציה SendLocalTime().

```
void SendLocalTime()
{
    char TimeString[5];

    Serial.print("~T=");
    sprintf(TimeString, "%02d: %02d", Hour, Minute);
    Serial.println(TimeString);
}
```

לאחר זמן רב גילינו שחסר תא במערך של המשתנה המקומי TimeString, כיוון שפונקציה sprintf אומנם מייצרת 5 תווים של HH:MM אך סוף של string הוא null שמצריך תו נוסף. לאחר הגדלת המערך התקלה נפתרה. אנו מעריכים שכיוון שהיה חסר תו שהיה במערך, פונקציית sprintf כתבה למחסנית שם נמצאים המשתנים המקומיים, ויתכן ודרסה את כתובת החזרה של הפונקציה, הפונקציה חזרה למקום לא ידוע, אך הפסיקה שהיא בכתובת קבועה המשיכה לעבוד.

6.1.3 פונקציית delay() אינה עובדת בתוך פסיקה

בזמן בדיקות המערכות השונות השתמשנו בפונקציית delay() להשהיות שונות, כששילבנו בתוכנת GreenDo שקוראת לחלק מהפונקציות מתוך פסיקה פונקציות delay לא ביצעו השהייה. וכך למשל אם היה צריך לפתוח ברז מים חשמלי, אספקת מתח לפין enable לדרייבר של המנוע התרחש מיד ללא השהייה ולכן הברז שצריך פולס של לפחות 100mSec לא נפתח.

הפתרון היה להשתמש בטיימרים תוכנתיים עם פונקציות callback כמתואר ב- 5.1.3.

6.1.4 Big/Little-endian

הארדוואינו וה-XBee שולחים מידע אחד לשני. המידע נשלח byte אחר byte. חלק מהמידע מייצג נתון של 16bit לדוגמה נתון A/D.

ה- XBee עובד בפורמט Big-endian כלומר MSByte first, ואילו הארדואינו עובד בפורמט Little-endian, כלומר LSByte first. גילינו בעיה זו כאשר קיבלנו ערכים מוזרים של A/D. השווינו לערכים שרואים ב- XCTU והבנו את הבעיה.

6.1.5 חישה טמפרטורה מציג ערכים לא נכונים

בבדיקות ראשונות של חישה הטמפרטורה המדידות היו הגיוניות. לאחר אינטגרציה והכנסה של מעגל החישה לתוך החממה דרך פלאג DB25, הטמפרטורה הייתה נמוכה ב-20° מהטמפרטורה האמיתית. בדקנו את המתח על רגל החישה ועל רגל הארדואינו וראינו שבארדואינו מקבלים מתח נמוך יותר בכ-0.1V. כשחיברנו את החישה עם חוטי jumper wires ללא מעבר דרך הפלאג, קיבלנו תוצאות תקינות. לא ברורה לנו הסיבה, אך יתכן וההלחמות בפלאג או הפלאג עצמו גרמו לבעיה.

6.1.6 קפיצה בנתון הטמפרטורה

בתנאים שונים, למשל לאחר למשל הפעלת מאווררים או מסיבות אחרות לא ברורות, חישה הטמפרטורה לפעמים קפצה למעלה לערכים לא הגיוניים, למשל 65°C ואח"כ חזרה לטמפרטורה הנכונה. הוספנו קבל קרמי של 0.1μF בין פין Vs+ של החישה לבין פין GND שלו, בהתאם להמלצת היצרן בדף הנתונים. ההוספה של הקבל שיפרה מעט, אך עדיין היו קפיצות מוזרות.

בזמן השוואת מדידה של טמפרטורת החישה לטמפרטורה של מד טמפרטורה מעבדתי ראינו שבחומם מד הטמפרטורה המעבדתי מגיב לאט לשינויים, ואילו החישה שלנו מגיב מהר. החלטנו להוסיף פילטר תוכנתי ע"י מיצוע של 10 דגימות (כל שניה יש דגימה אחת) וגם אם מתקבלת קריאה ששונה משמעותית מהממוצע האחרון, היא נזרקת ולא נוספת לממוצע. שינוי זה של פילטר תוכנתי שיפר את יציבות מדידת הטמפרטורה. בנוסף הלחמנו את כל החישים במעגל פרוטוטיפ במקום מטריצה, דבר שהוסיף ליציבות כי כל תזוזה של חוט השפיעה על המדידה.

6.1.7 טיימר 3 משפיע על PWM

Enable של דרייבר דלת הכניסה מחובר ל- PWM2 בארדואינו. בבדיקות של כל חלק בנפרד הפעלנו את ה- EN עם PWM ב- duty cycle של 50% כלומר 128, על מנת להאט את מהירות פתיחה וסגירת הדלת. באינטרגציה של התוכנה הדלת לא נפתחה ומסגרה כלל, והיינו חייבים לבצע EN ע"י digitalWrite. לא הצלחנו להבין את מקור התקלה אך הבנו שקשור לטיימר 3 ולכן השארנו את המצב כפי שהוא במשך מס' שבועות. לאחר מס' שבועות חיפשנו באינטרנט קשר בין PWM לטיימרים ומצאנו שהטיימרים משפיעים על תדר ה- PWM. מיפוי הטיימרים

timer 0 for pin 13 and 4

timer 1 for pin 12 and 11

timer 2 for pin 10 and 9

timer 3 for pin 5 and 3 and 2

timer 4 for pin 8 and 7 and 6

לפי המיפוי טיימר 3 משפיע על פין 2 ולכן הדלת לא פעלה כפי שצריך. בעקבות כך שינינו את טיימר הפסיקות מטיימר 3 לטיימר 4, וזה פתר את הבעיה.

6.2 קשיים בפרויקט

6.2.1 הלחמה ו-wire-wrap

היה לנו קשה לבצע הלחמות ו-wire-wrap בפרויקט, ולקח זמן רב עד שהצלחנו ללמוד איך לבצע זאת בצורה סבירה.

6.2.2 לימוד רכיב XBee

לימוד רכיב XBee לקח זמן רב כיוון שהנושא חדש לנו, ספר המשתמש של ה XBee מכיל עמודים רבים, ולקח שבועות רבים עד שהצלחנו להפעיל את הרכיב.

מה שעזר ללימוד הייתה תוכנת XCTU של ספק הרכיב. חיברנו 2 מודולים למחשבים שונים ובאמצעות התוכנה תקשרנו בין המחשבים ולמדנו בצורה זו את הרגיסטרים הנחוצים לנו לצורך הפרויקט. ישנם נושאים רבים ומורכבים ב-XBee שלא נגענו בהם, כיוון שלא היה צורך.

נושא קשור נוסף הוא פרוטוקול התקשורת בין ה-XBee לארדואינו, פרוטוקול API זה הצריך לא מעט קוד של בניית הודעות לשליחה ל-XBee מצד אחד וקבלת הודעות מה-XBee ופיענוחן מצד שני.

6.2.3 בניית דגם החממה

בניית הדגם של החממה, מחומרים פלסטיים ומתכת הייתה משימה שלא הצלחנו לעמוד בה ונאלצנו להיעזר בה באנשים אחרים לרבות מסגר לבניית גג החממה והתריס. למרות הקושי ושעות העבודה הרבות שהשקענו אנו מרוצים מאוד מהתוצאה הסופית.

6.2.4 פסיקות, טיימר תוכנתי ותוכנת GreenSee

כל נושאי הפסיקות, טיימר תוכנתי ושימוש ובניית תוכנה ב-C# WPF, היו חדשים לנו ולא ידענו איך לבצעם. בנושאים אלו קיבלנו ייעוץ ועזרה מאביו של אילון. וכיום אנו יכולים להגיד שנושאים אלו ברורים לנו ולמדנו רבות מיישומם בפרויקט.

6.2.5 האינטגרציה של המערכות

לאחר שבדקנו כל מערכת בנפרד, נתקלנו בקושי של אינטגרציה של המערכות. בחלק מהמקרים היו תקלות תוכנה כפי שפורט קודם לכן, וחלק היו בעיות חומרה. גם לאחר שהמערכות עבדו, היו מקרים שבהם עשינו שינויים ודברים הפסיקו לעבוד, ולעיתים הצריכו מספר שעות כדי למצוא את מקור הבעיה ולפתור אותה.

7 ביבליוגרפיה

- <https://www.arduino.cc/> - דף הבית של הארדואינו
- הסבר על עקרון פעולה של ממסר -
<https://he.wikipedia.org/wiki/%D7%9E%D7%9E%D7%A1%D7%A8>
- דף נתונים של חיישן הטמפרטורה -
https://www.analog.com/media/en/technical-documentation/datasheets/TMP36_37.pdf
- אתר הבית של יצרן הטנסיומטר -
<https://www.amitens.co.il/newpage9>
- מדריך ארדואינו -
<https://hackstore.co.il/books/arduino-for-beginners-1/#1>
- עקרון זיקת חום ע"י ונטה -
<http://www.t-vent.co.il/110284/%D7%95%D7%A0%D7%98%D7%94-%D7%9C%D7%92%D7%92>
- הסבר XBee -
<https://www.4project.co.il/product/xbee-pro-s1-wire-antenna>
- מדריך לחיבור דרייבר מנוע לארדואינו -
<https://tronixlabs.com.au/news/tutorial-l298n-dual-motor-controller-module-2a-and-arduino/>
- דף נתונים LDR -
<https://www.sunrom.com/p/ldr-light-dependent-resistor-gl5528-5mm>
- הסבר על פרוטוקול תקשורת טורי --
https://en.wikipedia.org/wiki/Universal_asynchronous_receiver-transmitter
- מדריך לחיבור ממסר לארדואינו -
<https://howtomechatronics.com/tutorials/arduino/control-high-voltage-devices-arduino-relay-tutorial/>
- הסבר על פורמט endianness -
<https://en.wikipedia.org/wiki/Endianness>
- מיפוי טיימרים לפינים של PWM -
<https://forum.arduino.cc/index.php?topic=72092.0>

8 **נספח א: תיעוד פגישות פרויקט**

טבלה 13: תיעוד פגישות פרויקט

תאריך	נושא הפגישה
23.11.18	ניסוי הזזת מנועים ע"י ארדואינו ב – tinkercad
2.12.18	ניסוי מנועי סרבו ע"י ארדואינו ב – tinkercad
8.12.18	למידת XBee מספר ההדרכה שלו
15.12.18	למידת XBee מספר ההדרכה שלו
22.12.18	למידת XBee מספר ההדרכה שלו
28.12.18	חיבור 2 מודולי XBee ל-2 מחשבים ובחינת שינוי הרגיסטרים, על מנת לקבל הודעות מ-XBee אחד לשני
29.12.18	חיבור טנסיומטר לרגל AD0 ב-XBee ושליחת הודעת I/O מ-XBee אחד לשני
2.1.19	בניית מעגל בקרת דלת וכתובת תוכנה לפתיחת הדלת בעזרת קודן
5.1.19	התחלת הגדרת ה-XBee ע"י שליחת פקודות API מהארדואינו ל-XBee
6.1.19	הפעלת מאוורר, הכנת מבנה נתונים ל-API, הכנת פונקציית Endian-Swap וכתובת פונקציה להכנה ושליחה של AT Commands
10.1.19	כתובת פונקציה לחישוב Checksum של הודעת API
16.1.19	התחלת בניית דגם החממה
30.1.19	ניסיון הפעלת המאווררים ישירות מהארדואינו ע"י PWM, והגעה למסקנה שצריך לשלוט על המאווררים דרך ממסר
1.2.19	חיבור הטנסיומטר ותחילת מדידות לחץ/יובש האדמה, הוספת אבטחה לתשדורת אלחוטית ע"י הוספת AES Key, הגדרת מחזור עבודה/שינה במעגל הצמח כדי לחסוך בסוללה
8.2.19	הפעלת מערכת דלת כניסה לחממה, חיבור הטנסיומטר למעגל הצמח וקבלת ערך המתח של הטנסיומטר וחיבור פאנל, מפסק ולדים למעגל הצמח
16.2.19	הגדרת טיימר חומרתי (טיימר 3 בארדואינו) שייתן פסיקה פעם בשנייה, הגדרת טיימרים תוכנתיים שכשכל אחד ברגע שפוקע מפעיל פונקציית callback, הגדרת בסיס נתונים של תוכנית ההשקיה והפעלה וסגירת הברז החשמלי בהתאם לתוכנית, ביצוע wire-wrap והלחמות למעגל של הצמח
15.3.19	התאמת צנרת מים לברז החשמלי ולמיכל המים וכתובת פונקציות לפתיחה וסגירת הברז ובדיקתן
16.3.19	התחלת כתיבת תוכנת GreenSee ותכנון פרוטוקול התקשורת בין הארדואינו למחשב
17.3.19	מימוש פרוטוקול התקשורת ובדיקת חיישן הטמפרטורה
30.3.19	המשך כתיבת תוכנת GreenSee ושליחת קונפיגורציה לארדואינו וכתובת תוכנה בארדואינו לפיענוח הקונפיגורציה

תאריך	נושא הפגישה
5.4.19	אינטגרציה של תוכנת GreenDo : צמח ופתיחת ברז
6.4.19	שילוב פתיחת דלת עם הקודן בתוכנת GreenDo
8.4.19	שרטוטי OrCAD
9.4.19	שרטוטי OrCAD
11.4.19	תחילת כתיבת ספר פרויקט
12.4.19	כתיבת ספר פרויקט
13.4.19	בניית דגם קונספט לחממה מקרטון
14.4.19	ניסיון לבניית דגם חממה מעץ, והחלטה שהדגם המתאים ייבנה מלוחות פוליגל
15.4.19	בניית דגם החממה מפוליגל
16.4.19	בניית דגם החממה מפוליגל
17.4.19	בניית דגם החממה מפוליגל
18.4.19	חיבור מערכות לדגם
19.4.19	בניית גג לדגם אצל מסגר והמשך חיבור מערכות לדגם
20.4.19	חיווט מערכות החממה למעגל הארדואינו
21.4.19	בדיקת כל מערכת בנפרד עם תוכניות בדיקה ייעודיות בארדואינו
22.4.19	אינטגרציית כל מערכות החממה בתוכנת GreeDo
23.4.19	פתרון תקלות שונות
24.4.19	פתרון תקלות שונות והמשך כתיבת הספר
25.4.19	סידור חוטים בחממה בתוך תעלות פלסטיק, הוספת מפסק למעגל החממה, הוספת מפסק ופוטנציומטר למעגל הצמח והמשך כתיבת הספר
26.4.19	המשך כתיבת הספר
1.5.19	סיום כתיבת ספר
3.5.19	פתרון בעיית השפעת טיימר 3 על PWM של פינים 2,3 הדבקת זוויות לחממה

תמונות משלבים בפרויקט



